

第7章 存储管理



第7章 存储管理



存储介质



存储引擎

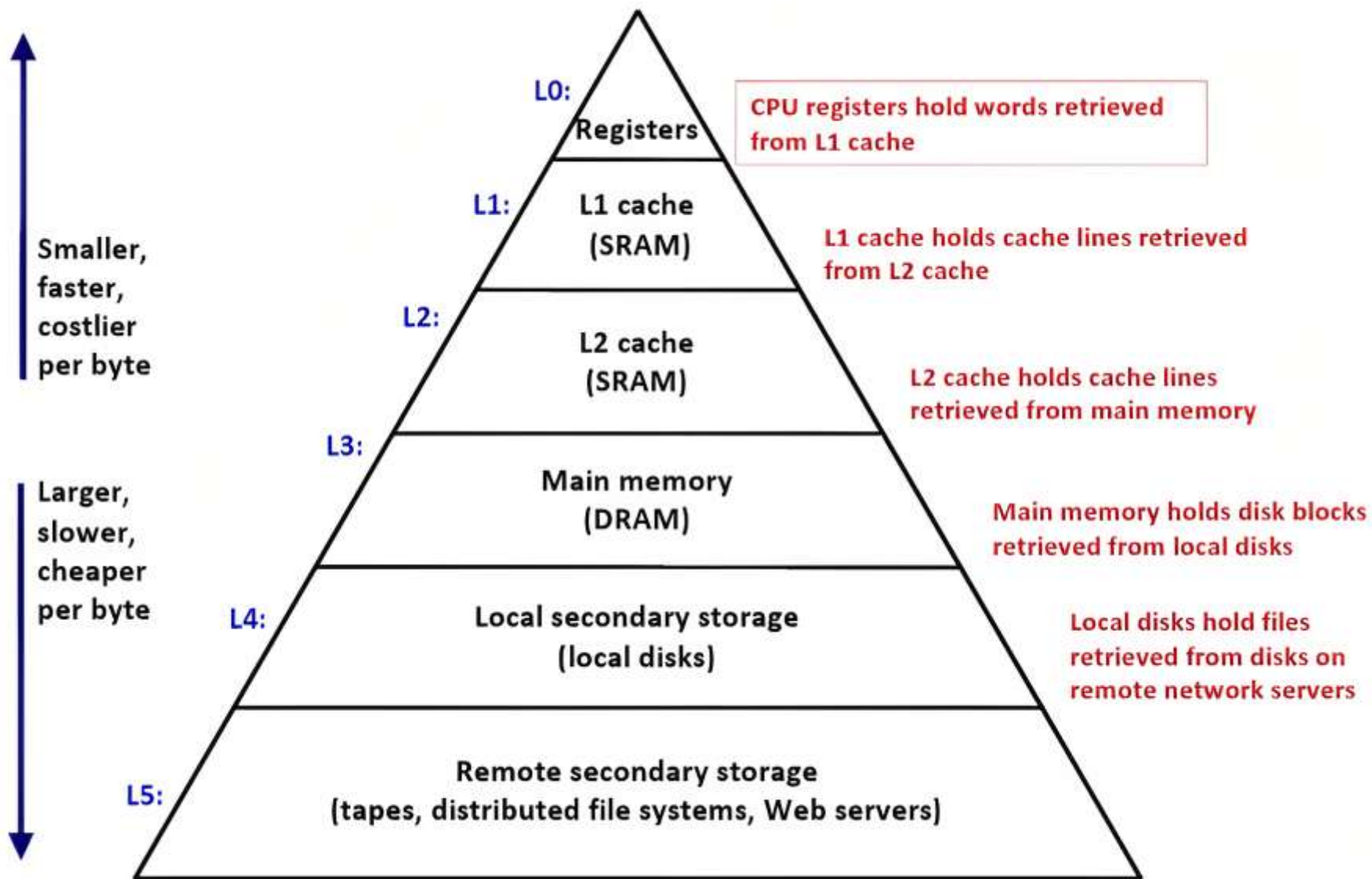


存储结构



缓冲区管理

■ 存储层次



■ 访问速度分类

- 主存储器
- 二级存储器
- 三级存储器

■ 操作分类

- 读操作
- 写操作

■ 联机分类

- 联机
- 脱机

■ 访问方式分类

- 随机访问
- 顺序访问

■ 读写单位分类

- 字节
- 块

■ 存储介质分类

➤ 易失存储器

易失性存储器中的数据在计算机重新启动（关机或崩溃后）时会丢失

- 主存储器

➤ 非易失存储器

非易失性存储器中的数据在计算机重新启动时不会丢失

- 二级存储器
- 三级存储器

■ 主存储器

使用 load/store 指令，主存储器中的数据可以被 CPU 直接操作

- 寄存器
- 高速缓存
- 内存

主存储器按字节寻址

■ 二级存储器

二级存储器中的数据不能被 CPU 直接处理，必须通过读/写指令先被复制到主存储器中才能被处理。

➤ 磁盘：机械硬盘 HDD

➤ 闪存：固态硬盘 SSD

二级存储器是联机的、按块寻址的

■ 三级存储器

三级存储器中的数据必须先被复制到二级存储器中

- 磁带
- 光盘
- 网络存储

三级存储器是脱机的、按块寻址的

■ 访存时间

Access time (ns)	Storage media
0.5	L1 cache
7	L2 cache
100	DRAM
150,000	SSD
10,000,000	HDD
30,000,000	Network storage
1,000,000,000	Tape

■ 数据传输级别



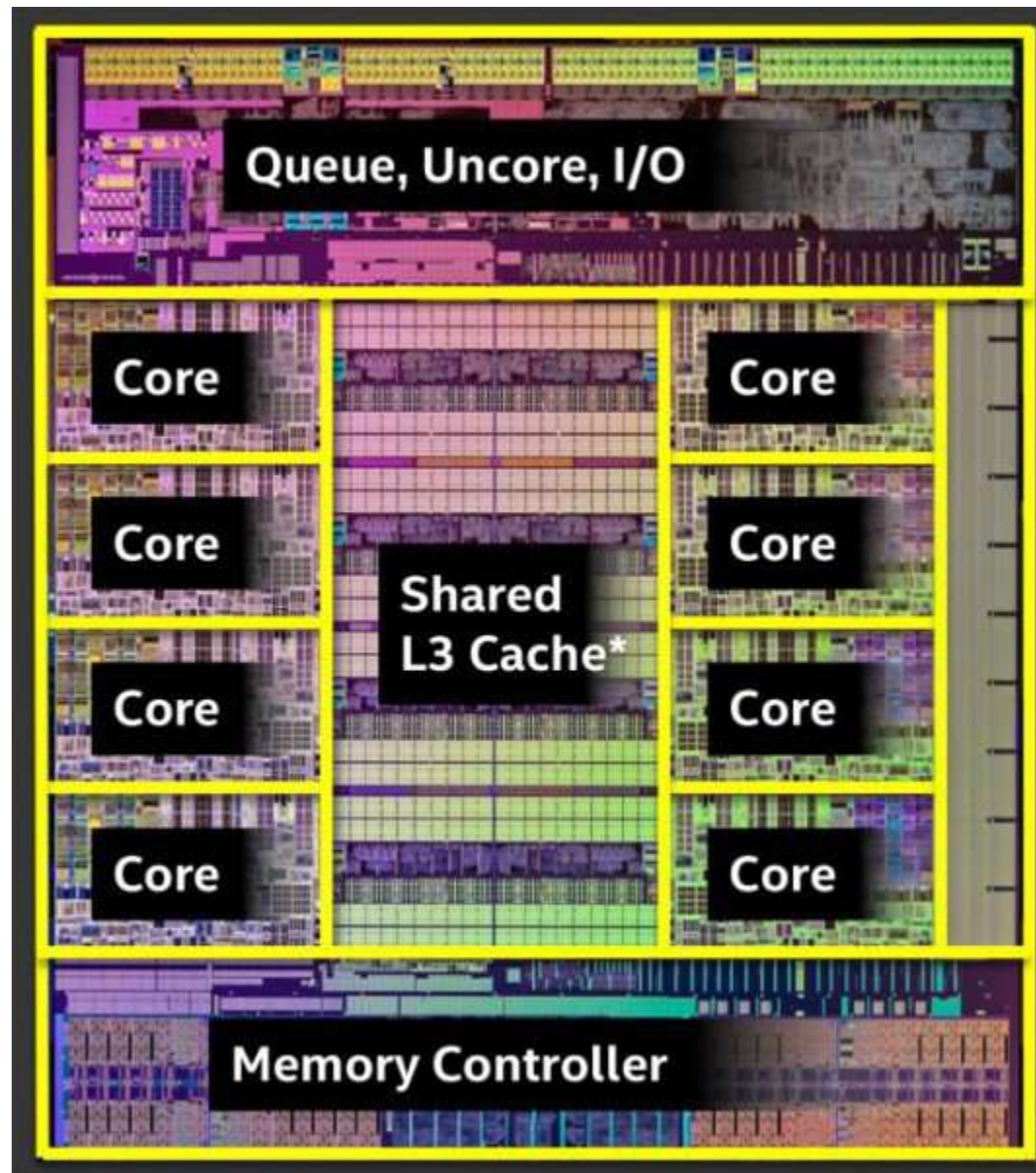
■ 寄存器 Register

CPU 中的寄存器是一种高速存储器件，用于存储和操作临时的数据和指令。不同 CPU 可能会有不同数量的寄存器，且不同架构的 CPU，寄存器的功能和作用可能会有所不同。

寄存器可以根据其功能和作用分以下几种：

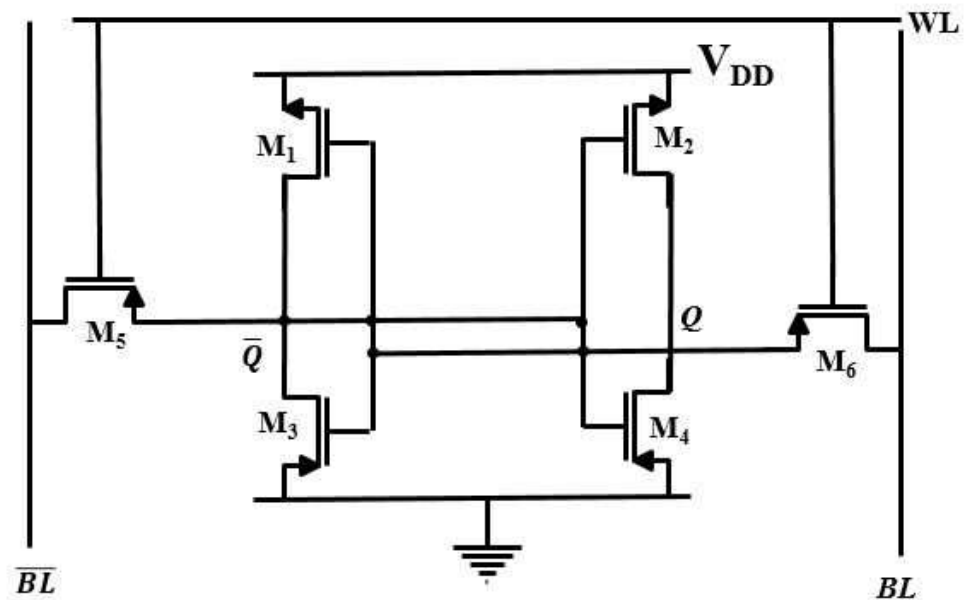
- 程序计数器（PC）：存储当前指令地址，执行指令后会自动加一，以便 CPU 能够顺序执行下一条指令
- 累加器（ACC）：用于保存运算的结果和中间值
- 数据寄存器（DR）：用于存储输入和输出的数据或者临时存放一些读取到的数据
- 地址寄存器（AR）：保存内存中被读取或写入的数据的地址
- 标志寄存器（FR）：存储和操作一些状态标志，如进位标志、零标志、溢出标志等
- 堆栈指针寄存器（SP）：保存堆栈中最后一个入栈的地址，以便维护堆栈的结构
- 指令寄存器（IR）：用于保存当前正在执行的指令

■ 缓存 Cache

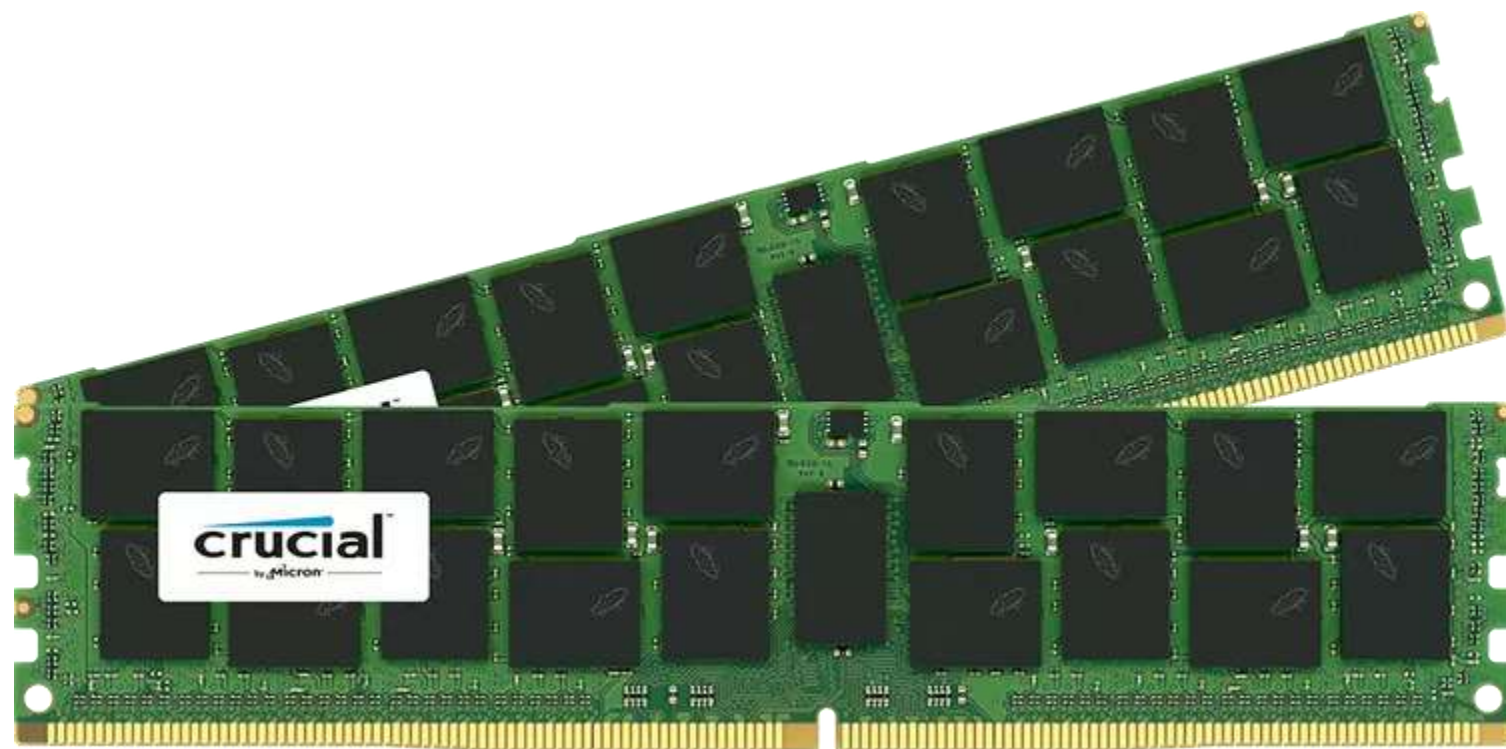


■ 静态随机访问存储 SRAM

- SRAM 是 Static Random Access Memory，静态随机访问存储器
- 保存一个 bit 需要 6 个晶体管，造价高
- SRAM 一般只有几个 MB 而已，再多了就不划算
- 从电路图可以看出，基本都是一些晶体管运算，速度很快
- SRAM 一般用来做高速缓存存储器
- SRAM 通常放在 CPU 芯片上

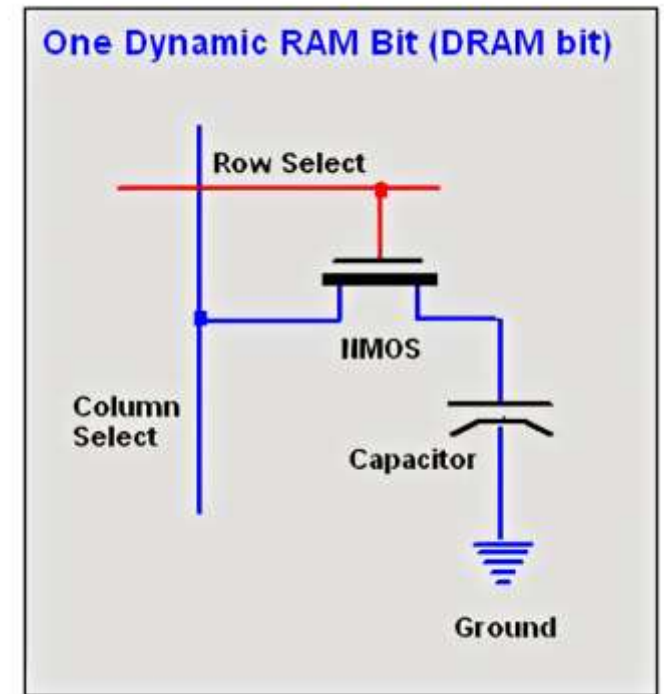


■ 内存



■ 动态随机访问存储 DRAM

- DRAM 是 Dynamic Random Access Memory，动态随机访问存储器
- DRAM 将每个比特存储在一个电容中
- DRAM 中的电容，其中储存的电荷会在几毫秒内流失
- 所以 DRAM 需要周期性刷新电容，所以被称为动态
- 因为只用到一个晶体管，所以成本低
- 刷新过程是在 DRAM 芯片内部完成的
- 不占用总线带宽或 CPU 时间
- 通常的刷新频率为 64ms 或者更短
- 以确保 DRAM 中的数据得到充分地保护且可靠读取
- 因需要刷新所以速度比 SRAM 慢



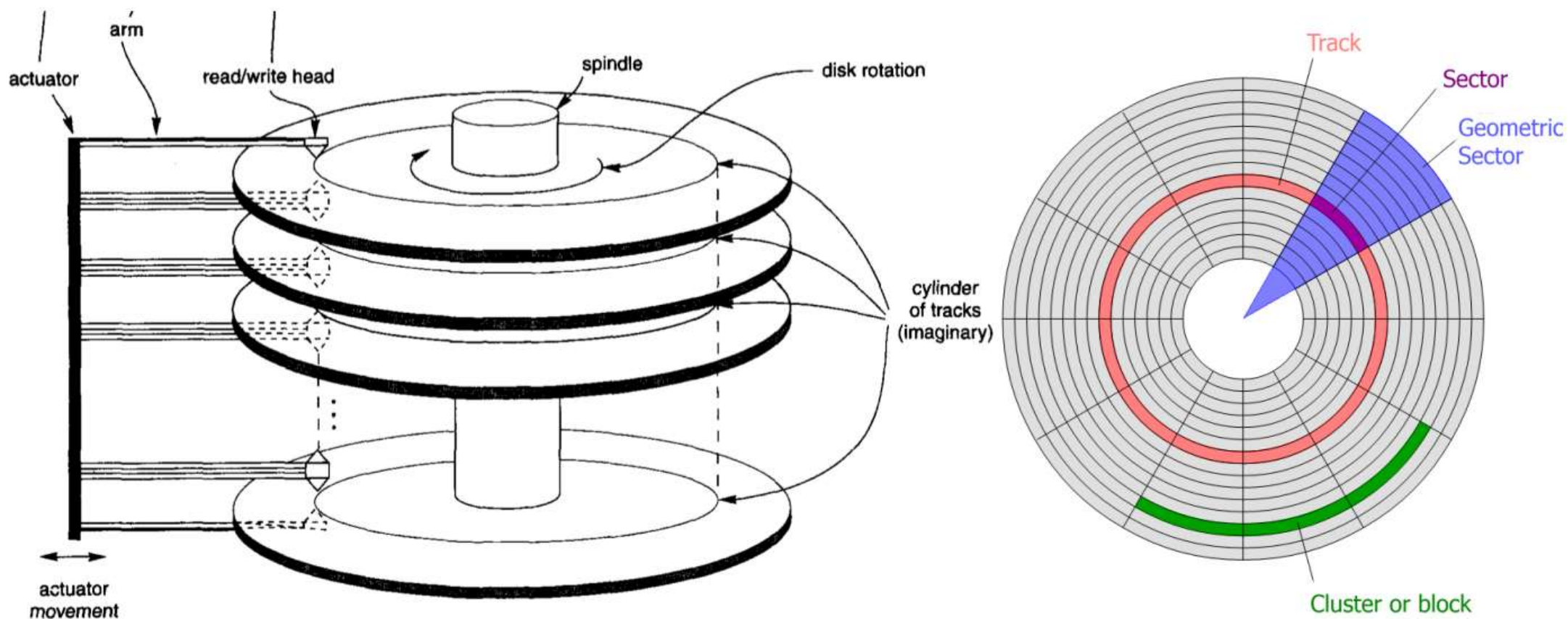
■ 机械硬盘 HDD (Hard Disk Drive)



■ 磁盘

磁盘有两个组成部分

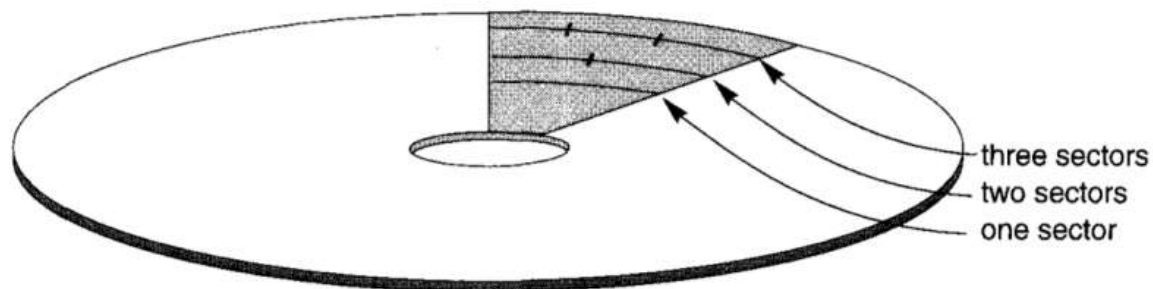
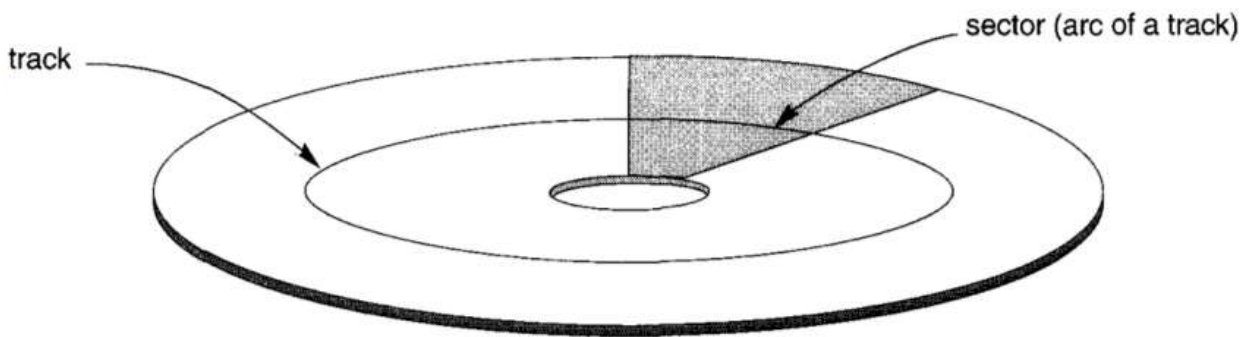
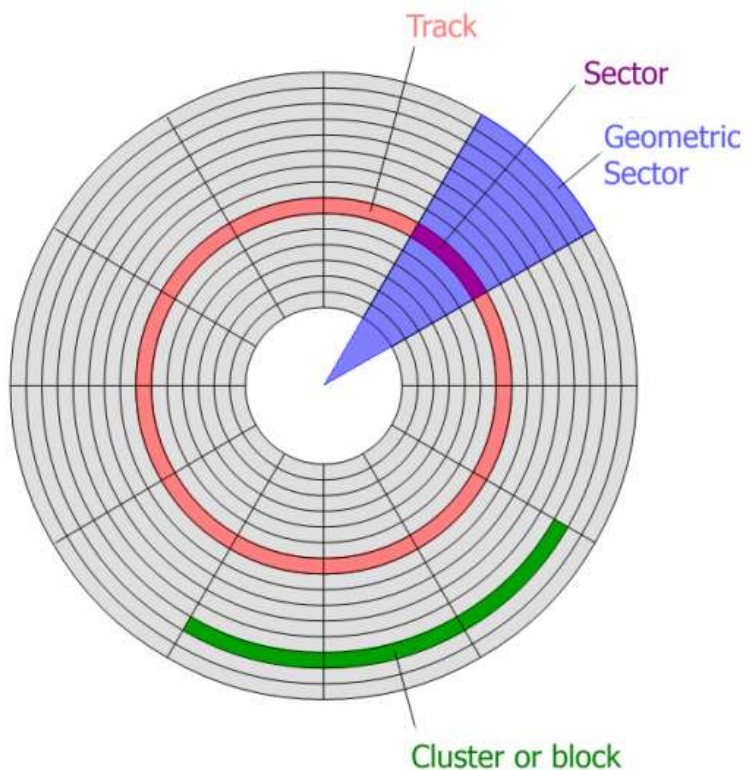
- 磁盘组件: sectors(扇区) $\subset$ tracks(磁道) $\subset$ cylinders(柱面)
- 磁头组件: disk heads(磁头), disk arms(磁臂)



■ 扇区

磁盘的每个磁道都被分成更小的扇区

- 将一个磁道分成扇区的方式已经硬编码在磁盘表面上，无法更改扇区的组织方式
- 方法1：扇区间距固定角度
- 方法2：扇区保持均匀的记录密度



■ 磁盘块/页

操作系统在磁盘格式化期间将一个磁道分成相等大小的磁盘块（或页）

- 一个磁盘块的大小可以设置为扇区大小的倍数
- 磁盘块的大小在磁盘格式化期间固定，不能动态改变
- 典型的磁盘块大小为 512B - 4KB

逻辑块地址（Logical Block Address, LBA）

磁盘块的逻辑块地址是一个在 0 到 $n-1$ 之间的数字（假设磁盘的总容量为 n 个块）

■ 读取速率

磁盘访问时间的三个重要参数

- 平均寻道时间
- 平均延迟时间, 取决于磁盘转速 7200 转/分钟
- 数据传输速率

磁盘访问方式

- 顺序访问
- 随机访问

■ 固态硬盘 SSD (Solid State Disk)



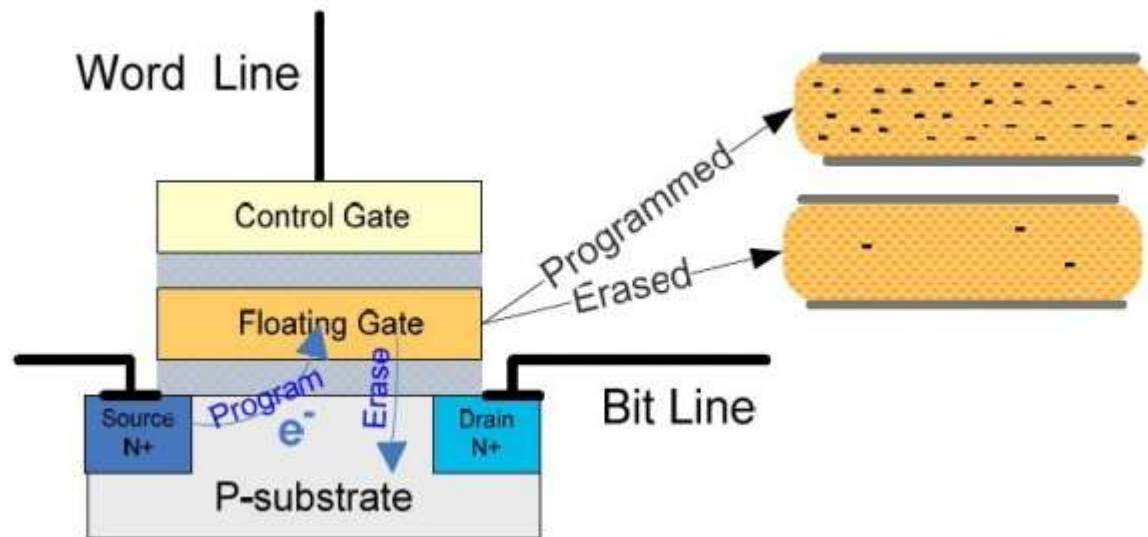
■ 闪存 flash memory

闪存是一种非易失性存储器，也称为固态存储器，具有以下特点：

- 高速读写：闪存的读写速度比机械硬盘快，因为它通过电子信号进行读写操作
- 耐用：闪存具有较长的使用寿命，并且不容易受到外界的磁场和震动的影响
- 体积小：闪存的内部结构比机械硬盘短小精悍，因其尺寸小，适合于小型电子设备
- 低功耗：闪存的电源消耗相对较低，非常适合移动设备和便携式电脑等应用

应用场景：

- 存储卡和 USB 存储设备
- 固态硬盘
- 移动电话和平板电脑
- 数码相机



■ 磁带 tape



■ 光盘 CD/DVD



■ 网络存储 NAS





存储介质



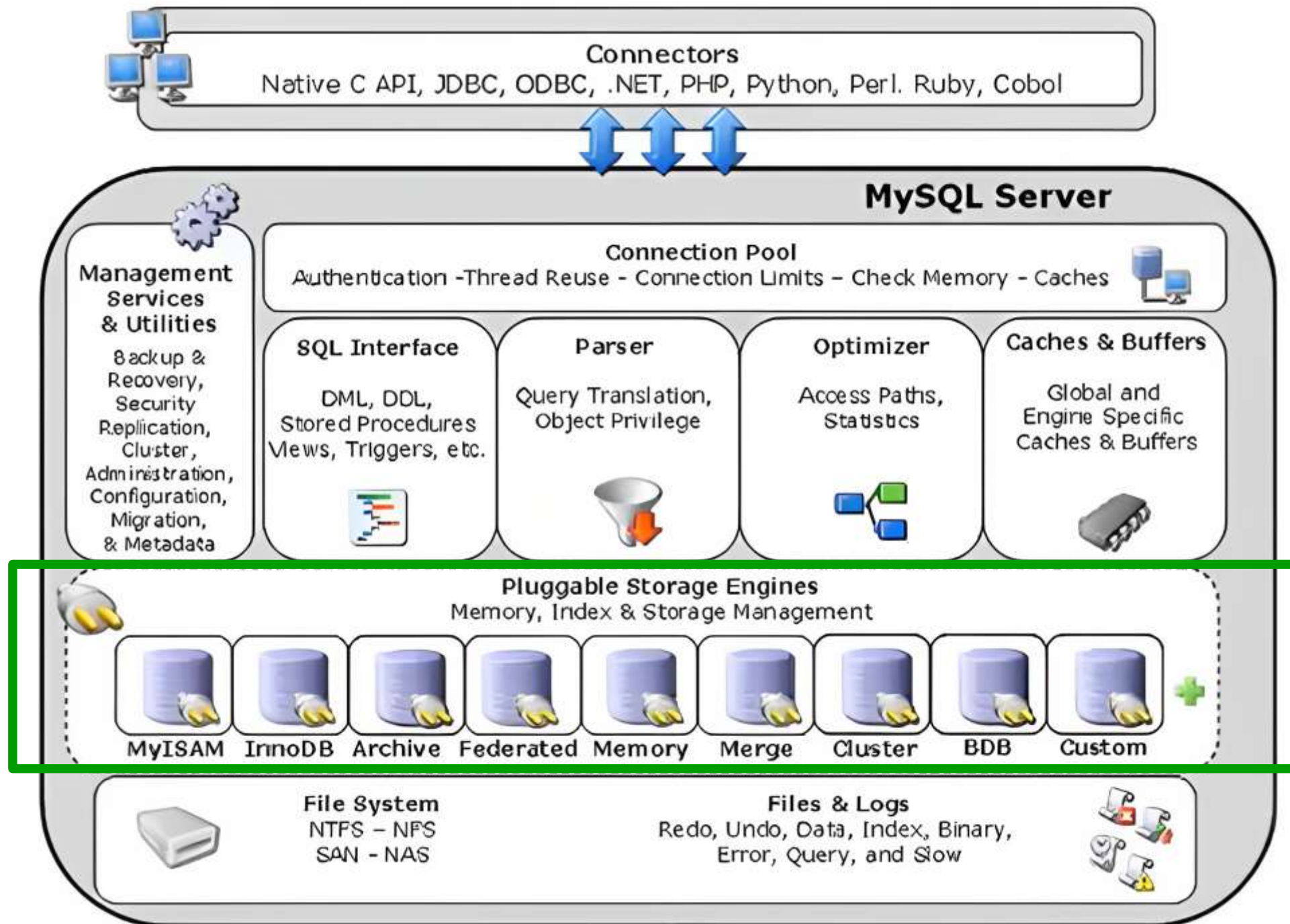
存储引擎

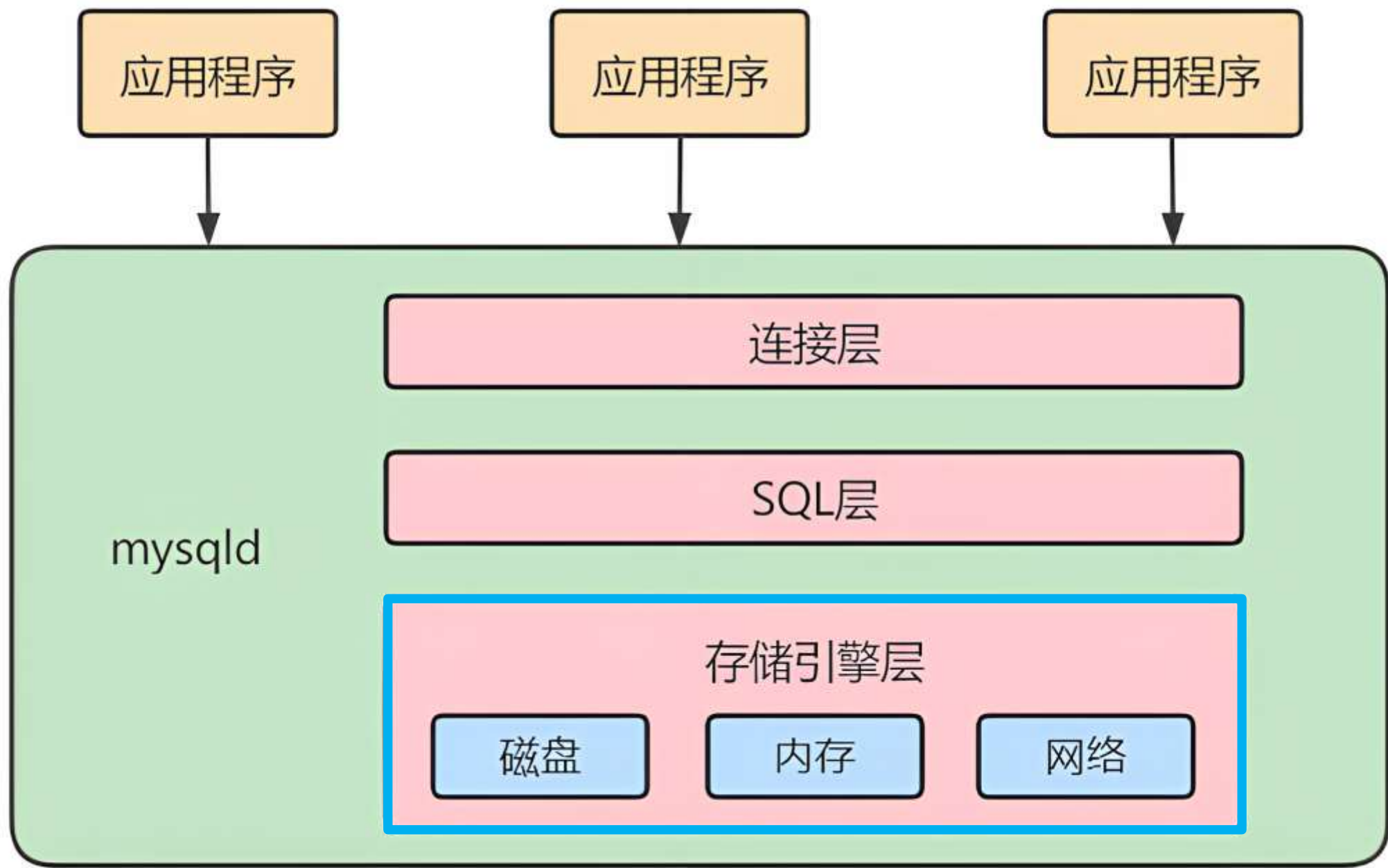


存储结构



缓冲区管理





■ 概述

- 存储引擎不同，数据在磁盘上的结构不同
- 存储引擎不同，索引可能不同
- MySQL 的存储引擎是可插拔的
- MySQL 的存储引擎是基于表的

■ 存储引擎种类

- InnoDB 引擎
- MyISAM 引擎
- Archive 引擎
- Blackhole 引擎
- CSV 引擎
- Memory 引擎
- Federated 引擎
- Merge 引擎
- NDB 引擎

■ 存储引擎的相关操作

- 查看 MySQL 支持的所有存储引擎
- 查看 MySQL 默认的存储引擎
- 查看某个表使用的存储引擎
- 创建表时指定存储引擎
- 修改表的存储引擎

■ InnoDB 存储引擎

- InnoDB 是 MySQL **默认的存储引擎**
- InnoDB 被设计用来处理大量的短期**事务**
- 如表中除了增加和查询外，还需要更新和删除，则应优先选择 InnoDB 存储引擎
- 除非有非常特别的原因需要使用其他的存储引擎，否则应该优先考虑 InnoDB 引擎
- 对比 MyISAM 的存储引擎，InnoDB **写的处理效率差一些**
- InnoDB **会占用更多的磁盘空间**以保存数据和索引
- MyISAM 只缓存索引，不缓存真实数据；InnoDB 不仅缓存索引还要缓存真实数据
- InnoDB **对内存要求较高**，而且内存大小对性能有决定性的影响

■ MyISAM 存储引擎

- MyISAM 提供了大量的特性，包括全文索引、压缩、空间函数(GIS) 等
 - MyISAM **不支持事务**、行级锁、外键，主要缺陷是崩溃后无法安全恢复
 - 优势是**访问速度快**，对事务完整性没有要求或者以 **SELECT、INSERT 为主的应用**
 - 针对数据统计有额外的常数存储。故而 count(*) 的查询效率很高
 - 应用场景：**只读应用或者以读为主的业务**
-
- **OLTP** OnLine Transaction Processing，联机事务处理
 - **OLAP** OnLine Analytics Processing，联机分析处理

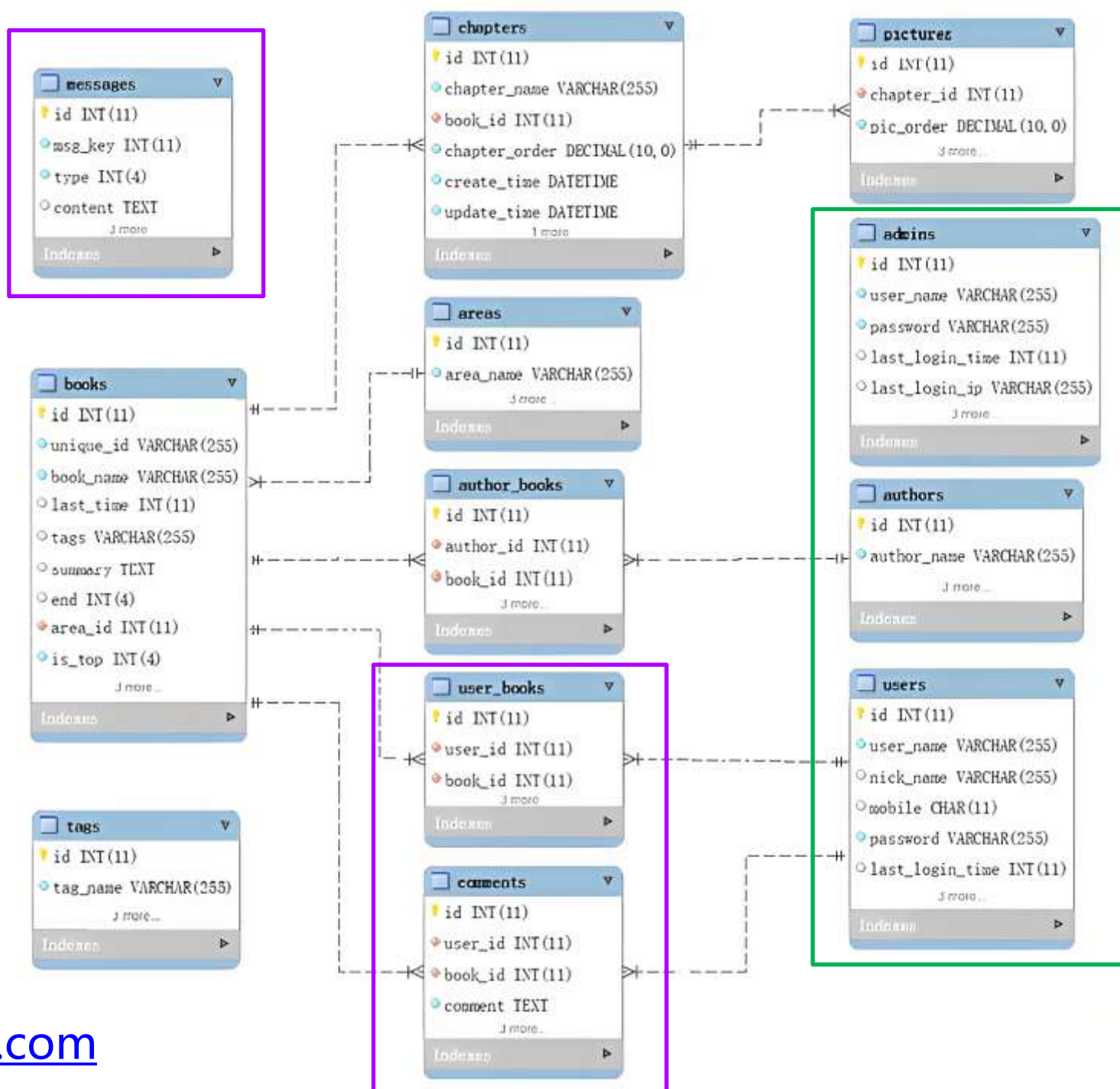
■ Memory 存储引擎

- Memory 采用的逻辑**介质是内存，响应速度很快**
- 当 mysqld 守护进程崩溃的时候**数据会丢失**
- 要求存储的数据是**数据长度不变**的格式，如，Blob 和 Text 类型的数据不可用
- Memory 同时支持哈希索引和 B+ 树索引
- Memory 表至少比 MyISAM 表要快一个数量级
- Memory 表的**大小受限制**
- 数据文件与索引文件分开存储
- 缺点：其数据易丢失，生命周期短

特点	MyISAM	InnoDB	Memory	Merge	NDB
存储限制	有	64TB	有	没有	有
事务安全	/	支持	/	/	/
锁机制	表锁	行锁	表锁	表锁	行锁
B+树索引	支持	支持	支持	支持	支持
索引缓存	只缓存索引 不缓存数据	缓存索引 缓存数据 比较占内存 访问速度快	支持	支持	支持

■ 案例

物理设计之选择存储引擎



动漫屋网站: <https://dm5.com>



存储介质



存储引擎



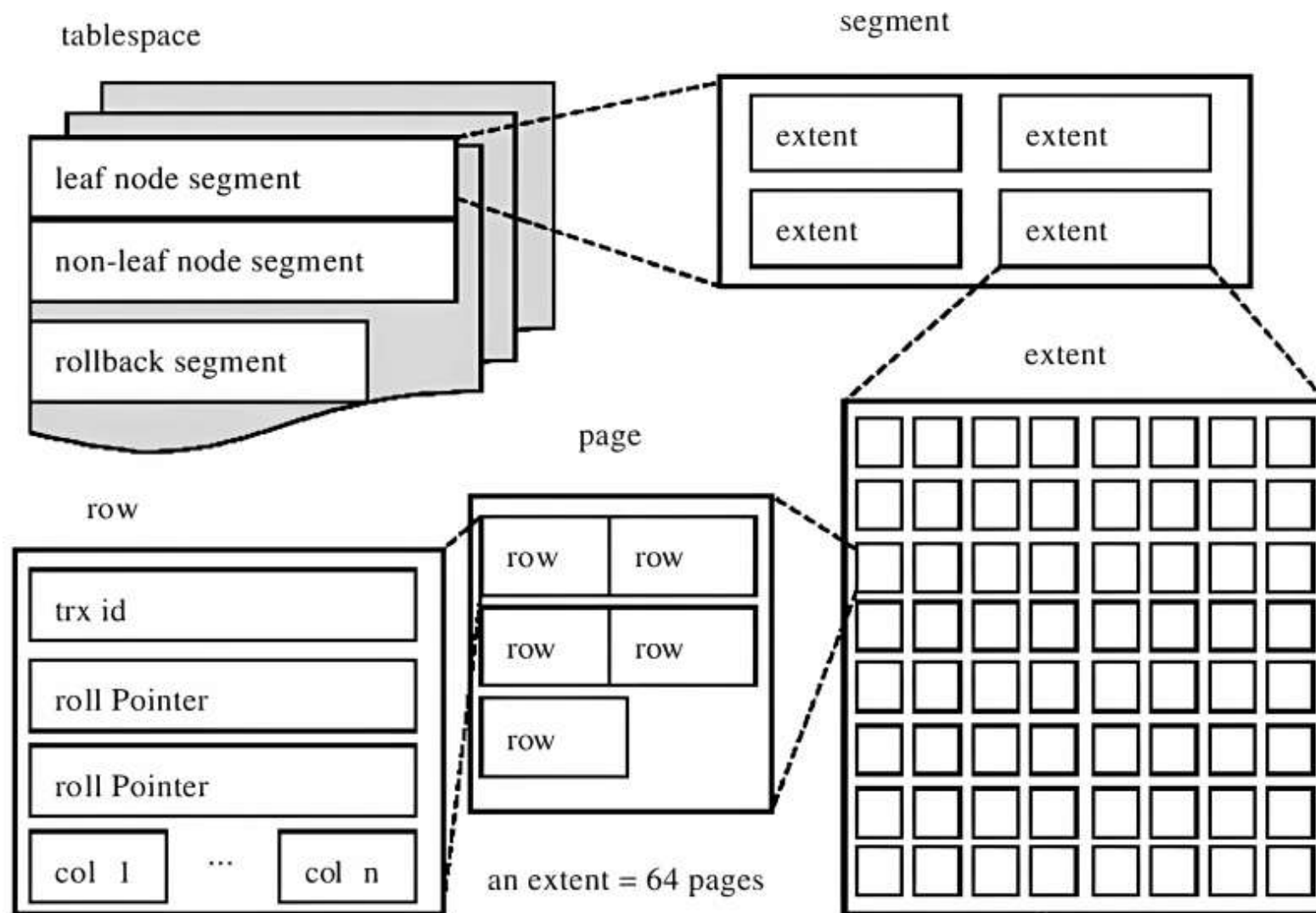
存储结构



缓冲区管理

■ 磁盘存储结构

- 表空间
- 段 segment
- 区 extent
- 页 page
- 行 row



系统表空间	/var/lib/mysql/ibdata1
用户表空间	/var/lib/mysql/ db_name / tb_name .ibd
临时表空间	/var/lib/mysql/ibtmp1

行结构

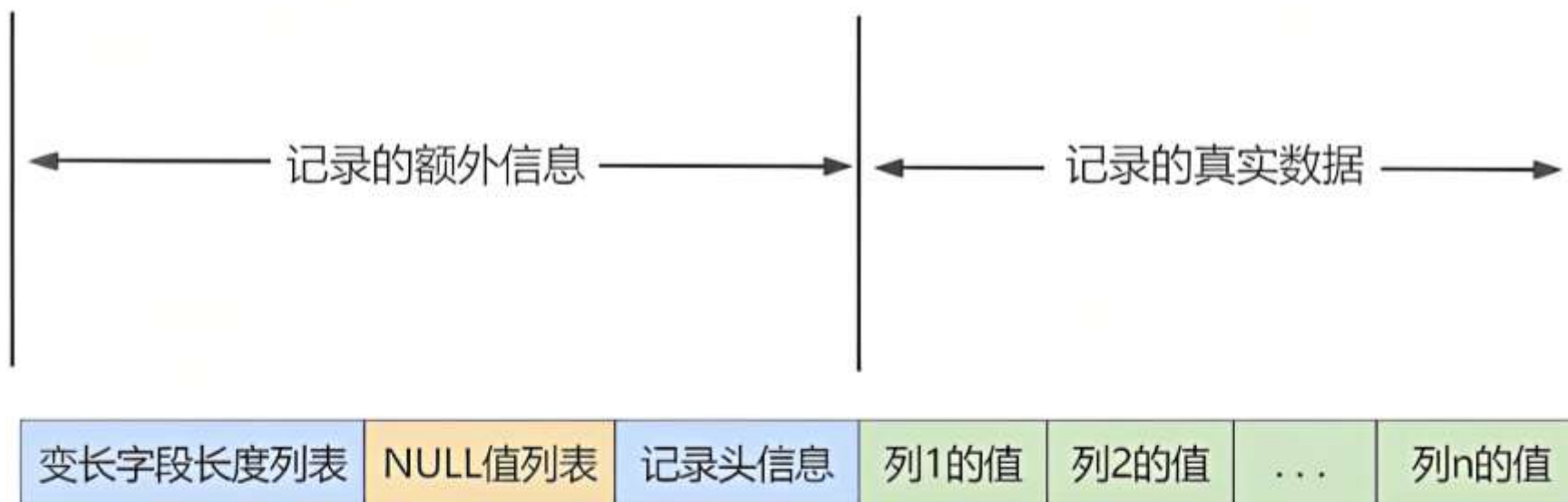
■ 行格式种类

- Compact 行格式
- Dynamic 行格式
- Compressed 行格式
- Redundant 行格式

■ 行格式的相关操作

- 查看 MySQL 默认的行格式
- 查看某个表的行格式
- 创建表时指定行格式
- 修改表的行格式

■ Compact 行格式



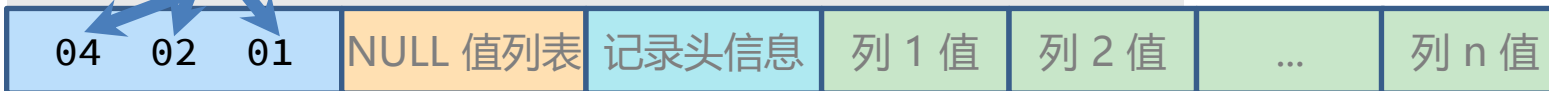
■ 变长字段长度列表

```
create table tb_compact (  
  col1 varchar(10),  
  col2 varchar(10),  
  col3 char(10),  
  col4 varchar(10)  
) charset=latin1 row_format=compact;
```

```
insert into tb_compact values  
( 'a', 'bb', 'ccc', 'dddd' ),  
( 'e', null, 'fff', null );
```

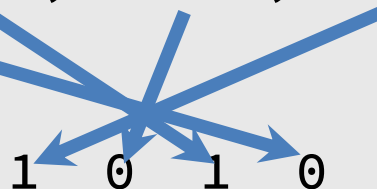
- 把所有变长字段真实数据占用的字节长度都存放在记录的开头
- 存储的长度和字段顺序是相反的

a	0x01
bb	0x02
dddd	0x04



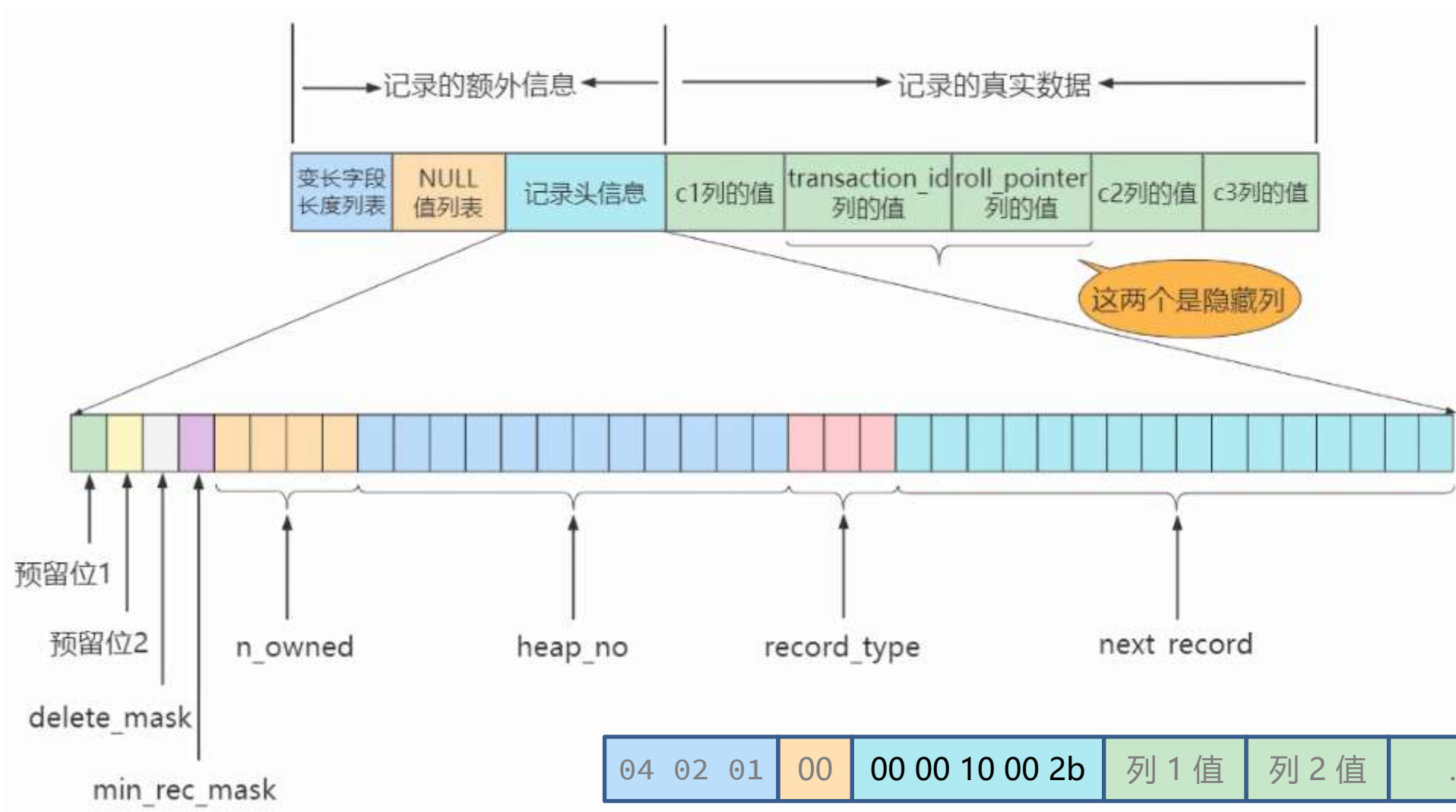
■ NULL 值列表

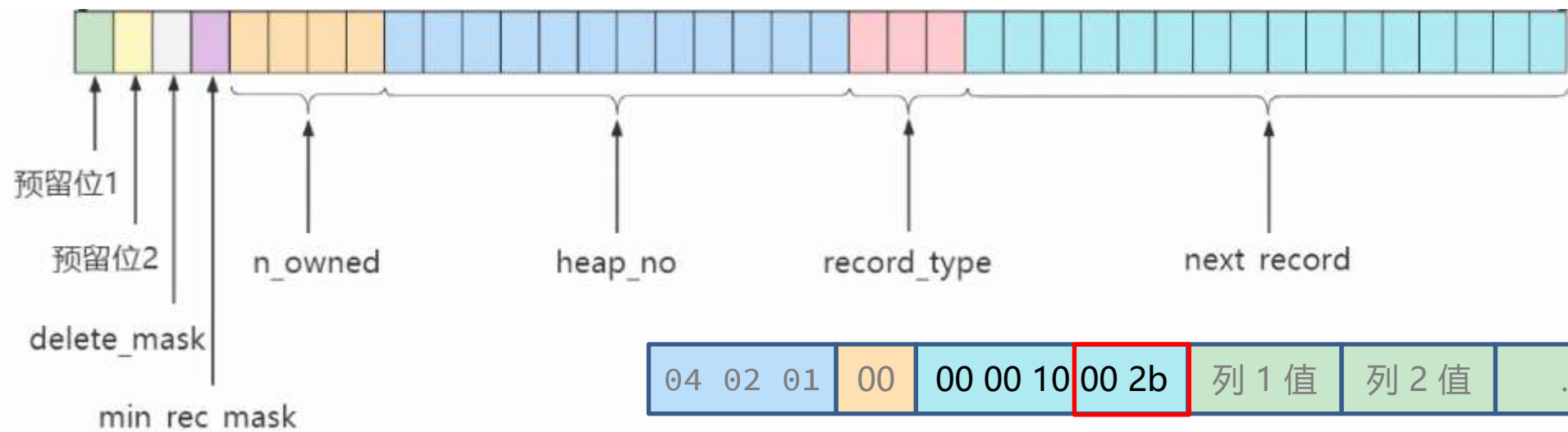
```
create table tb_cmpt (  
  col1 varchar(10),  
  col2 varchar(8),  
  col3 char(8),  
  col4 varchar(8)  
) charset=latin1 row_format=compact;  
insert into tb_compact values  
( 'a', 'bb', 'ccc', 'dddd' ),  
( 'e', null, 'fff', null );
```



- 统一管理可以为 NULL 的列，存成 NULL 值列表。如果表中没有可以为 NULL 的列，则 NULL 值列表不存在
- 二进制位的值为 1 时，代表该列的值为 NULL
- 二进制位的值为 0 时，代表该列的值不为 NULL
- 跟字段的顺序相反

■ 记录头信息（5字节）





名称	大小 (单位: bit)	描述
预留位1	1	没有使用
预留位2	1	没有使用
delete_mask	1	标记该记录是否被删除
min_rec_mask	1	B+树的每层非叶子节点中的最小记录都会添加该标记
n_owned	4	表示当前记录拥有的记录数
heap_no	13	表示当前记录在记录堆的位置信息
record_type	3	表示当前记录的类型, 0 表示普通记录, 1 表示B+树非叶节点记录, 2 表示最小记录, 3 表示最大记录
next_record	16	表示下一条记录的相对位置

最小记录：

delete_mask	min_rec_mask	n_owned	heap_no	record_type	next_record	
0	0	1	0	2		'infimum'

最大记录：

delete_mask	min_rec_mask	n_owned	heap_no	record_type	next_record	
0	0	4	1	3	0	'supremum'

第1条记录：

delete_mask	min_rec_mask	n_owned	heap_no	record_type	next_record				
0	0	0	2	0		1	100	'aaaa'	其他信息

第2条记录：

delete_mask	min_rec_mask	n_owned	heap_no	record_type	next_record				
1	0	0	3	0	0	2	200	'bbbb'	其他信息

这条记录被删了

第3条记录：

delete_mask	min_rec_mask	n_owned	heap_no	record_type	next_record				
0	0	0	4	0		3	300	'cccc'	其他信息

第4条记录：

delete_mask	min_rec_mask	n_owned	heap_no	record_type	next_record				
0	0	0	5	0		4	400	'dddd'	其他信息

■ 真实数据

MySQL 会为每个记录默认的增加一些列（也称为隐藏列），真实的列名是 DB_ROW_ID、DB_TRX_ID 和 DB_ROLL_PTR

列名	是否必须	占用空间	描述
row_id	否	6 字节	行ID, 唯一标识一条记录
transaction_id	是	6 字节	事务ID
roll_pointer	是	7 字节	回滚指针

第一条记录

04 02 01	00	00 00 10 00 2b	00 00 00 00 16 00	00 00 00 aa 3c 17	b1 00 00 02 4d 01 10
----------	----	----------------	-------------------	-------------------	----------------------

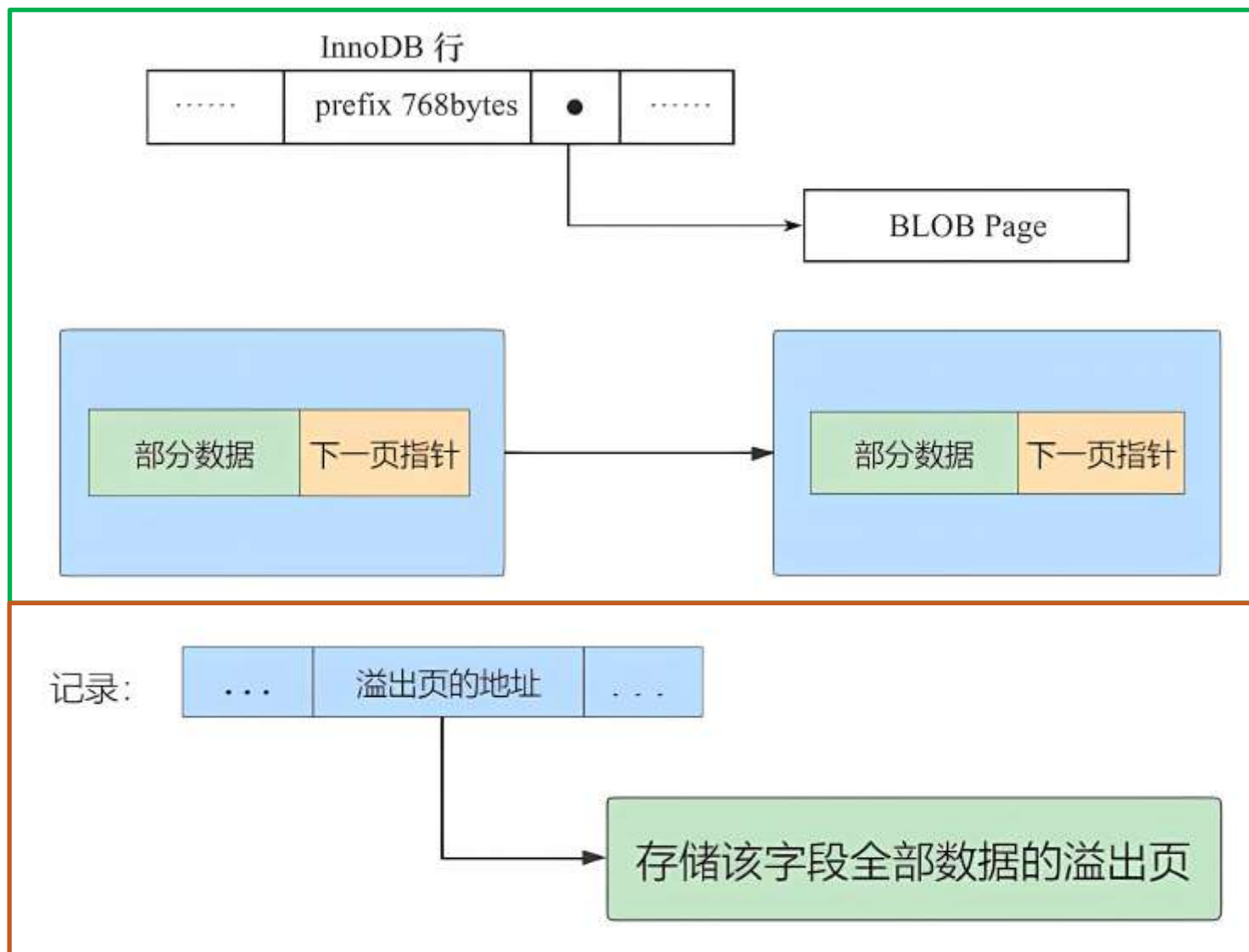
隐藏列

61	62 62	63 63 63 20 20 20 20 20 20 20	64 64 64 64
----	-------	-------------------------------	-------------

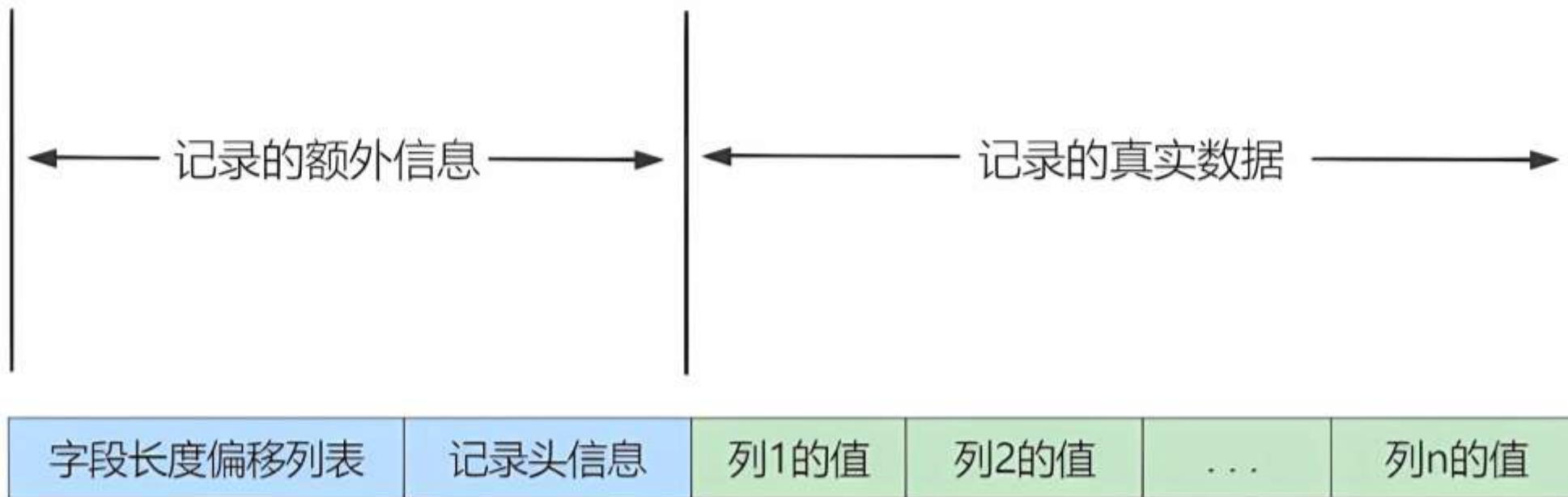
真实列 col1, ..., col4

■ Dynamic 和 Compressed 行格式

- 行溢出
- 数据压缩



■ Redundant 行格式



页结构

■ 页

页是 InnoDB 管理存储空间的基本单位，一个页的大小一般是 16KB。InnoDB 为了不同的目的设计了许多种不同类型的页。页是磁盘和内存数据交换的基本单位，也是 B+ 树索引节点的基本单位。

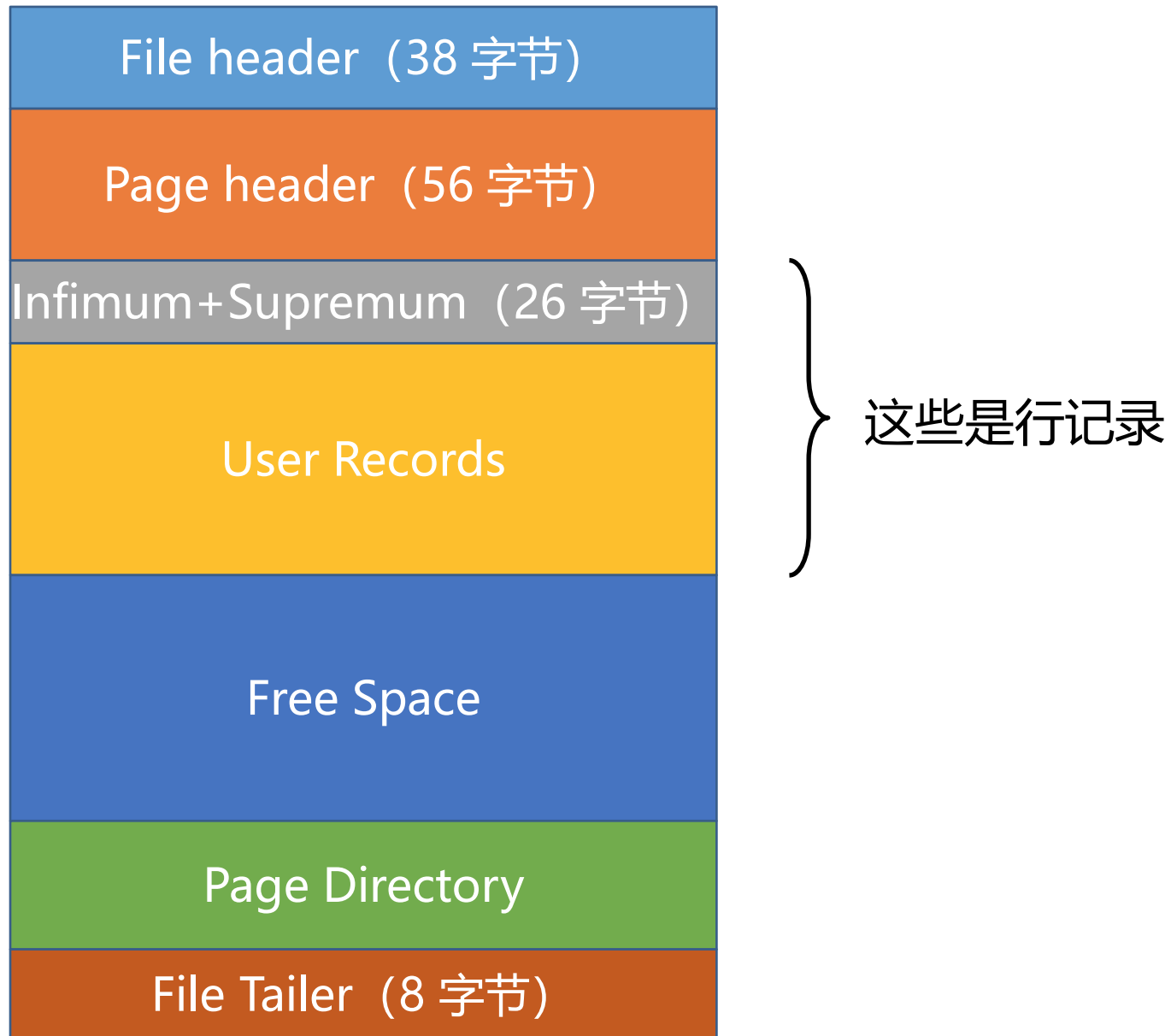
常见页的类型：

- 表空间头部信息页
- Insert Buffer 页
- inode 信息页
- undo 日志页
- 索引页（就是数据页），是记录的容器

■ 页相关操作

➤ 查看 MySQL 默认页尺寸

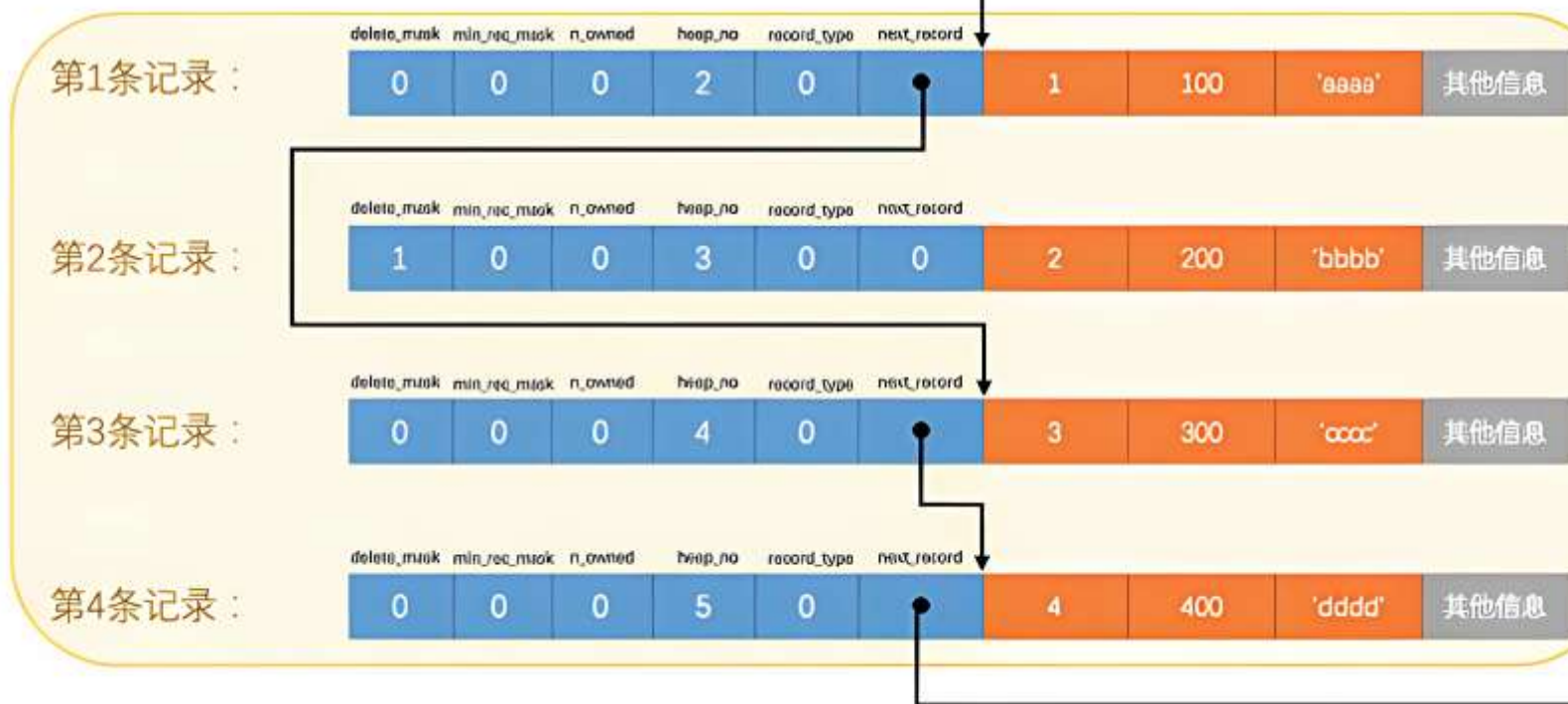
■ 索引页结构



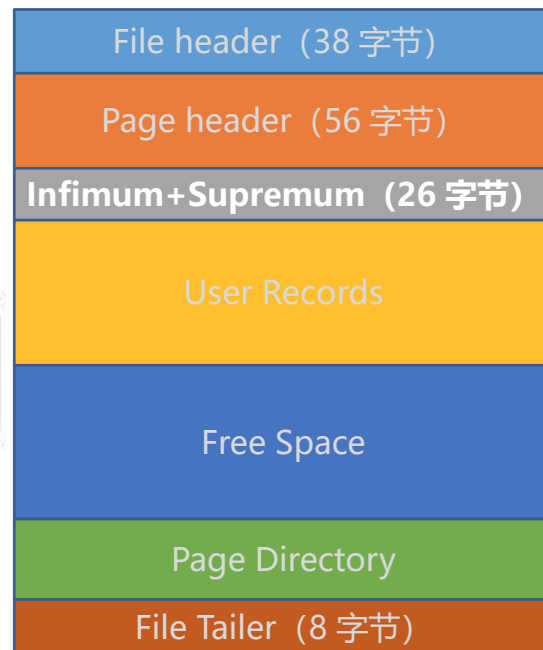
■ 索引页结构（续）

名称	中文名	占用空间大小	简单描述
File Header	文件头部	38 字节	页的一些通用信息
Page Header	页面头部	56 字节	数据页专有的一些信息
Infimum + Supremum	最小记录和最大记录	26 字节	两个虚拟的行记录
User Records	用户记录	不确定	实际存储的行记录内容
Free Space	空闲空间	不确定	页中尚未使用的空间
Page Directory	页面目录	不确定	页中的某些记录的相对位置
File Trailer	文件尾部	8 字节	校验页是否完整

■ 最大记录、最小记录

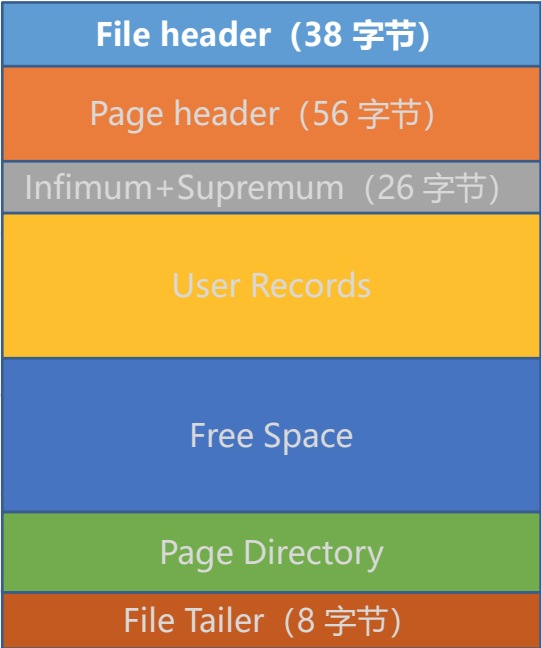


这条记录被删了



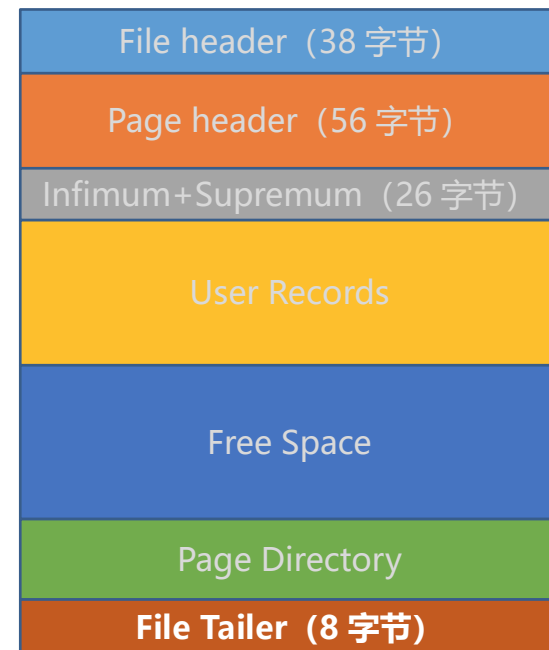
■ 文件头部结构 (38 字节)

名称	占用空间大小	描述
FIL_PAGE_SPACE_OR_CHKSUM	4 字节	页的校验和 (checksum值)
FIL_PAGE_OFFSET	4 字节	页号
FIL_PAGE_PREV	4 字节	上一个页的页号
FIL_PAGE_NEXT	4 字节	下一个页的页号
FIL_PAGE_LSN	8 字节	页面被最后修改时对应的日志序列位置 (英文名是: Log Sequence Number)
FIL_PAGE_TYPE	2 字节	该页的类型
FIL_PAGE_FILE_FLUSH_LSN	8 字节	仅在系统表空间的一个页中定义, 代表文件至少被刷新到了对应的LSN值
FIL_PAGE_ARCH_LOG_NO_OR_SPACE_ID	4 字节	页属于哪个表空间

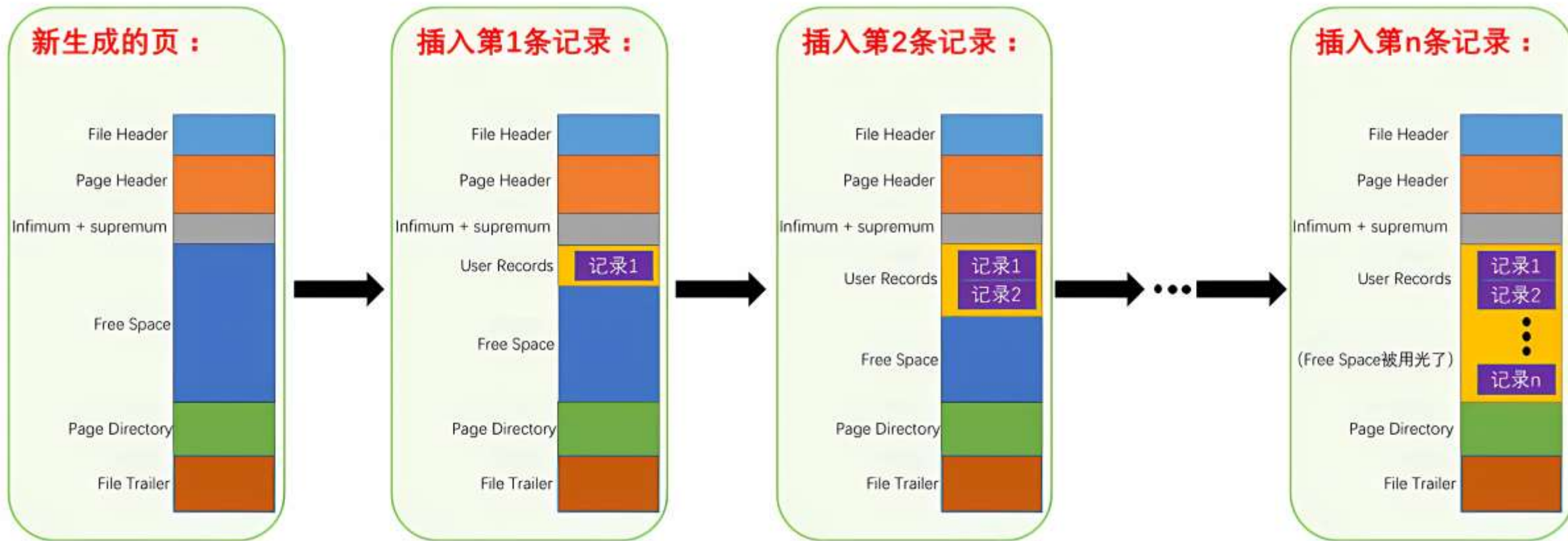


■ 文件尾部结构 (8 字节)

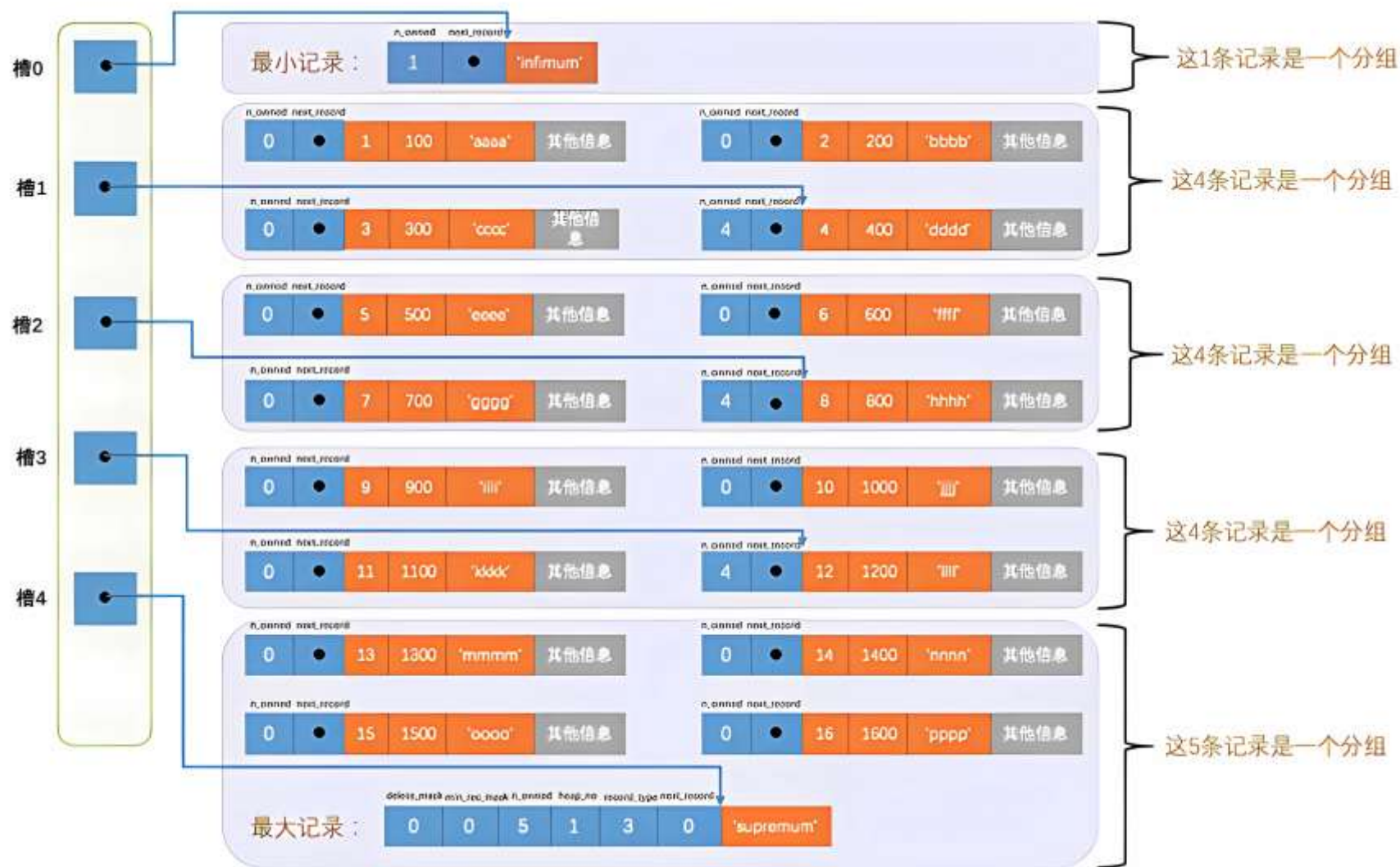
- 前 4 个字节代表页的校验和
这个部分与 File Header 中的校验和相对应。
- 后 4 个字节代表页面被最后修改时对应的日志序列位置 (LSN)
这个部分也是为了校验页的完整性，如果首部和尾部的 LSN 值校验不成功的话，就说明同步过程出现了问题。



■ 空闲空间、用户记录



■ 页目录结构



File header (38 字节)
Page header (56 字节)
Infimum+Supremum (26 字节)
User Records
Free Space
Page Directory
File Tailer (8 字节)

■ 页头结构

名称	占用空间大小	描述
PAGE_N_DIR_SLOTS	2字节	在页目录中的槽数量
PAGE_HEAP_TOP	2字节	还未使用的空间最小地址，也就是说从该地址之后就是
PAGE_N_HEAP	2字节	本页中的记录的数量（包括最小和最大记录以及标记为删除的记录）
PAGE_FREE	2字节	第一个已经标记为删除的记录地址（各个已删除的记录通过 next_record 也会组成一个单链表，这个单链表中的记录可以被重新利用）
PAGE_GARBAGE	2字节	已删除记录占用的字节数
PAGE_LAST_INSERT	2字节	最后插入记录的位置
PAGE_DIRECTION	2字节	记录插入的方向
PAGE_N_DIRECTION	2字节	一个方向连续插入的记录数量
PAGE_N_RECS	2字节	该页中记录的数量（不包括最小和最大记录以及被标记为删除的记录）

File header (38 字节)

Page header (56 字节)

Infimum+Supremum (26 字节)

User Records

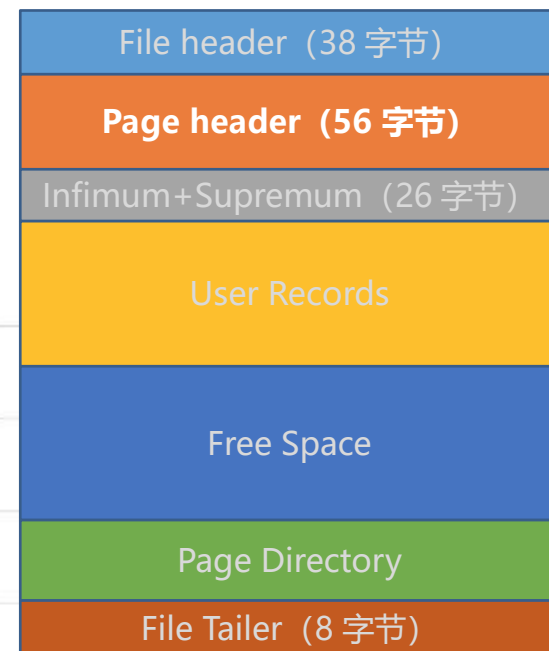
Free Space

Page Directory

File Tailer (8 字节)

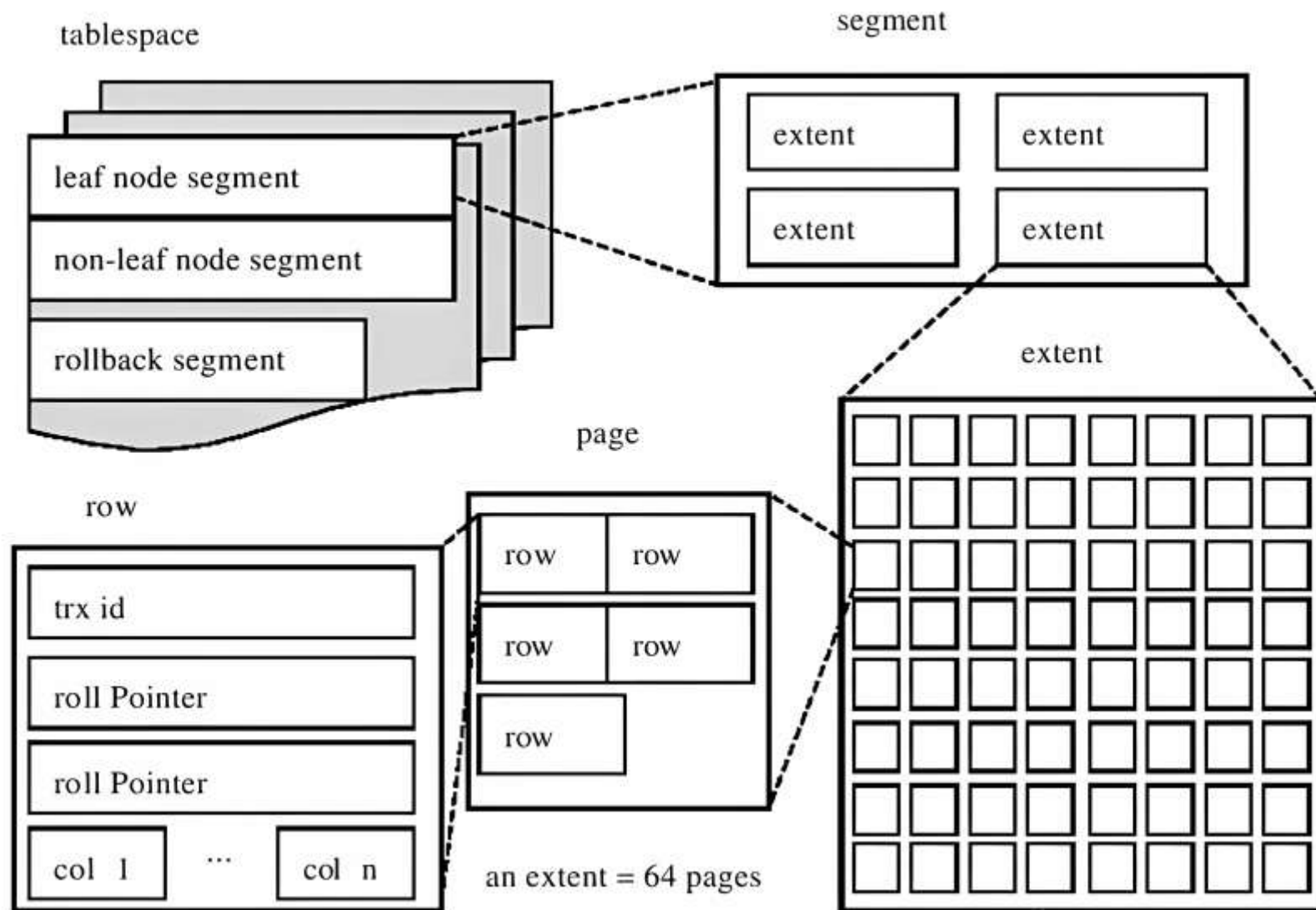
■ 页头结构 (续)

PAGE_MAX_TRX_ID	8字节	修改当前页的最大事务ID, 该值仅在二级索引中定义
PAGE_LEVEL	2字节	当前页在B+树中所处的层级
PAGE_INDEX_ID	8字节	索引ID, 表示当前页属于哪个索引
PAGE_BTR_SEG_LEAF	10字节	B+树叶子段的头部信息, 仅在B+树的Root页定义
PAGE_BTR_SEG_TOP	10字节	B+树非叶子段的头部信息, 仅在B+树的Root页定义



■ 磁盘存储结构

- 表空间
- 段 segment
- 区 extent
- 页 page
- 行 row



■ 表空间相关操作

- 查看系统表空间和临时表空间的路径设置
- 查看是否启用独立表空间（用户表空间）

系统表空间 /var/lib/mysql/ibdata1

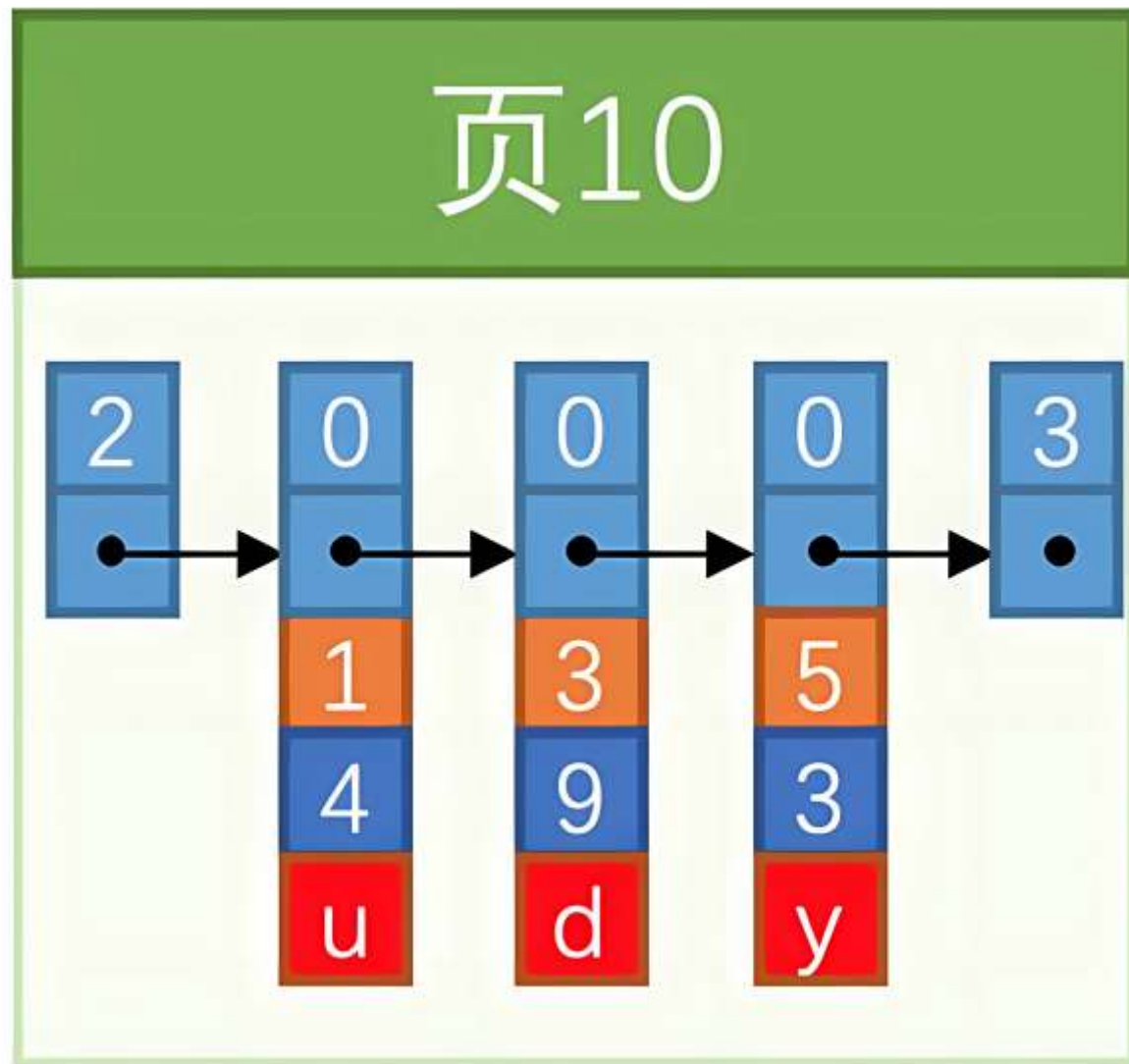
用户表空间 /var/lib/mysql/**db_name**/**tb_name**.ibd

临时表空间 /var/lib/mysql/ibtmp1

B + 树和页结构

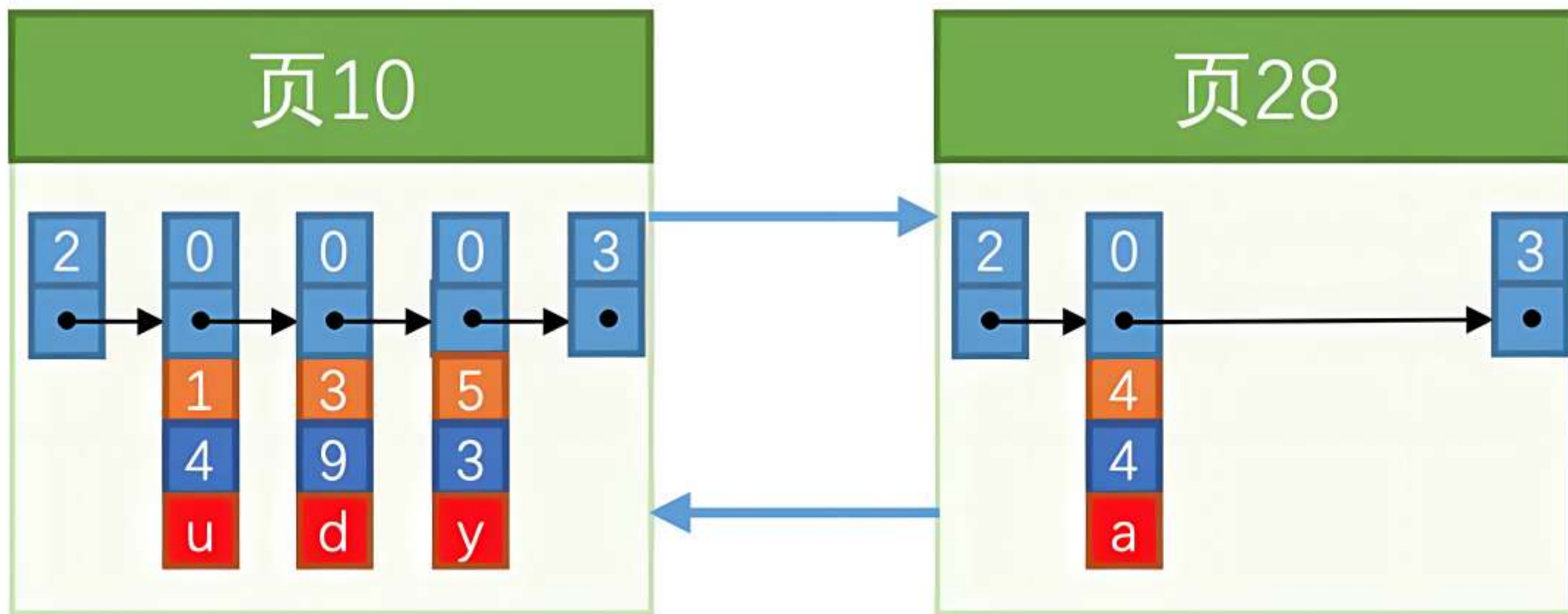
■ page 示意图

```
create table tb_idx (  
  c1 int,  
  c2 int,  
  c3 char,  
  primary key(c1)  
)row_format=compact;  
insert into tb_idx values  
(1, 4, 'u'),  
(3, 9, 'd'),  
(5, 3, 'y');
```

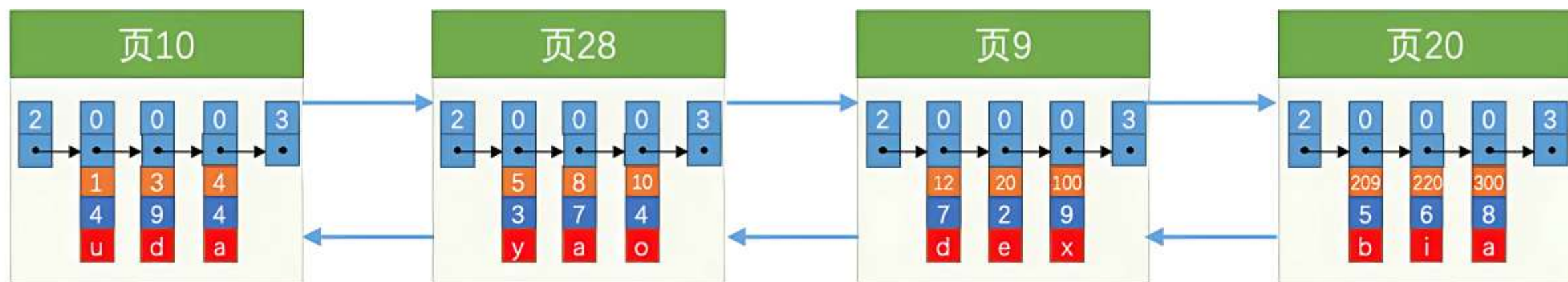


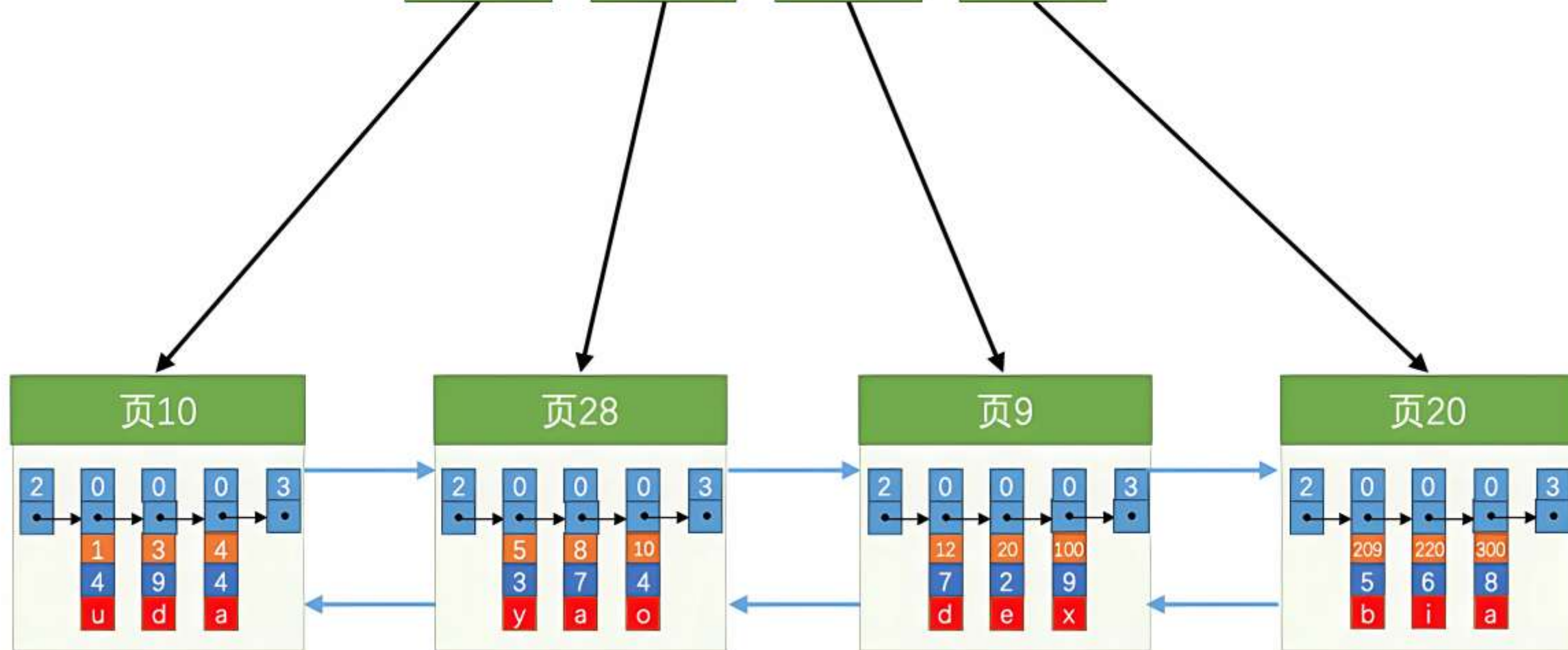

```
mysql> INSERT INTO index_demo VALUES(4, 4, 'a');  
Query OK, 1 row affected (0.00 sec)
```

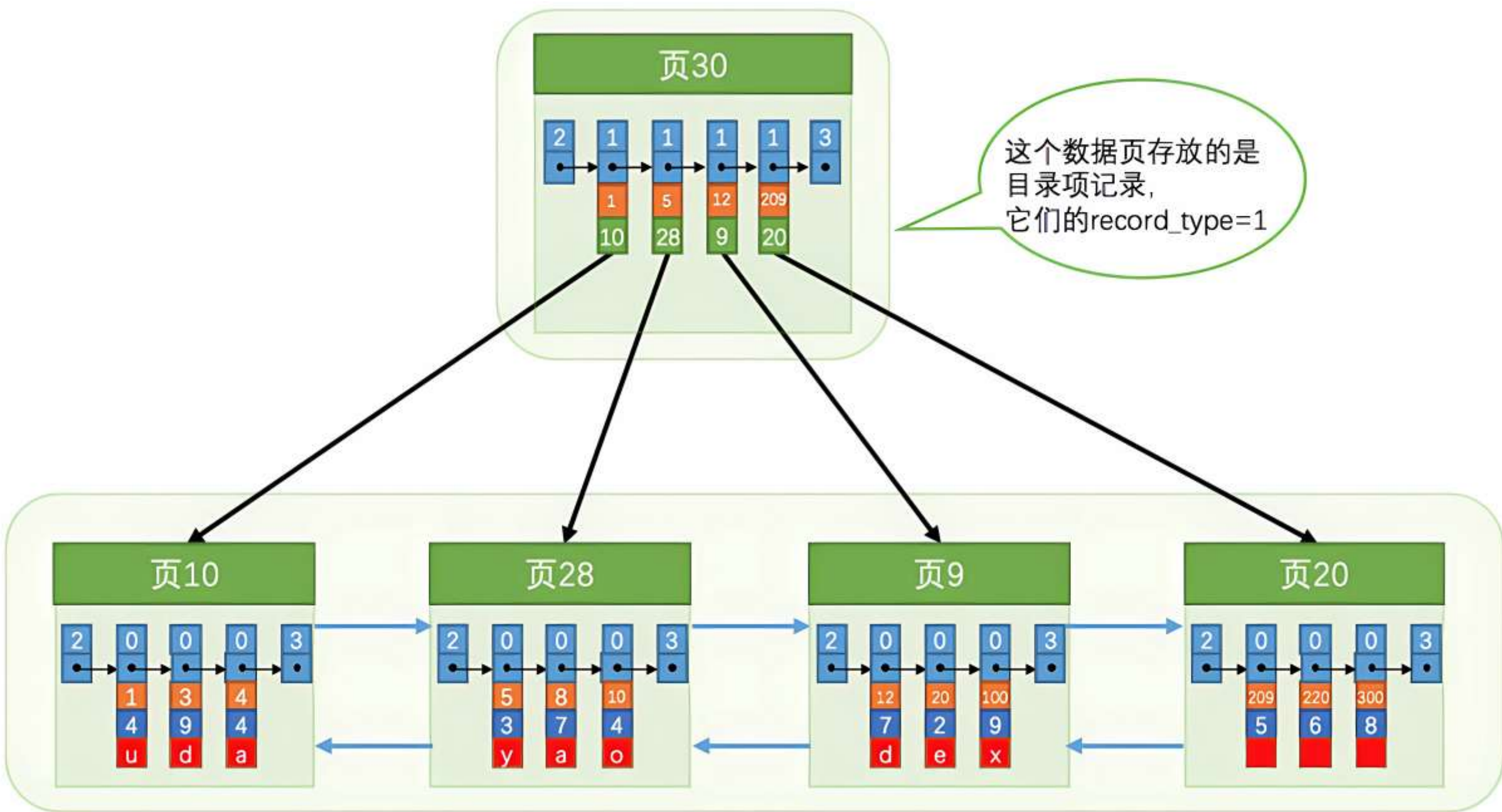
因为 页10 最多只能放3条记录，所以我们~~不得不再~~分配一个新页：



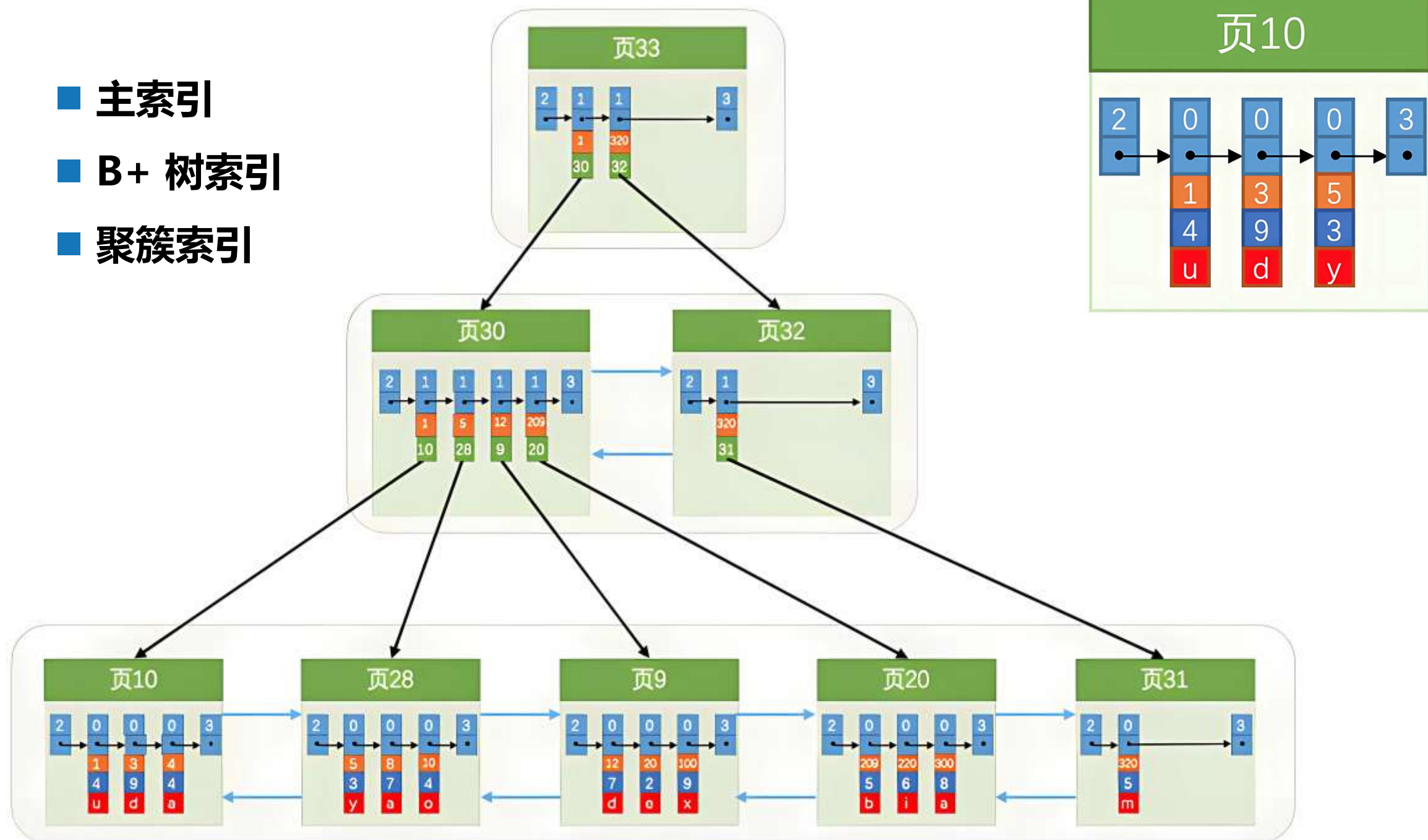
由于数据页的编号可能并不是连续的，所以在向 index_demo 表中插入许多条记录后，可能是这样的效果：







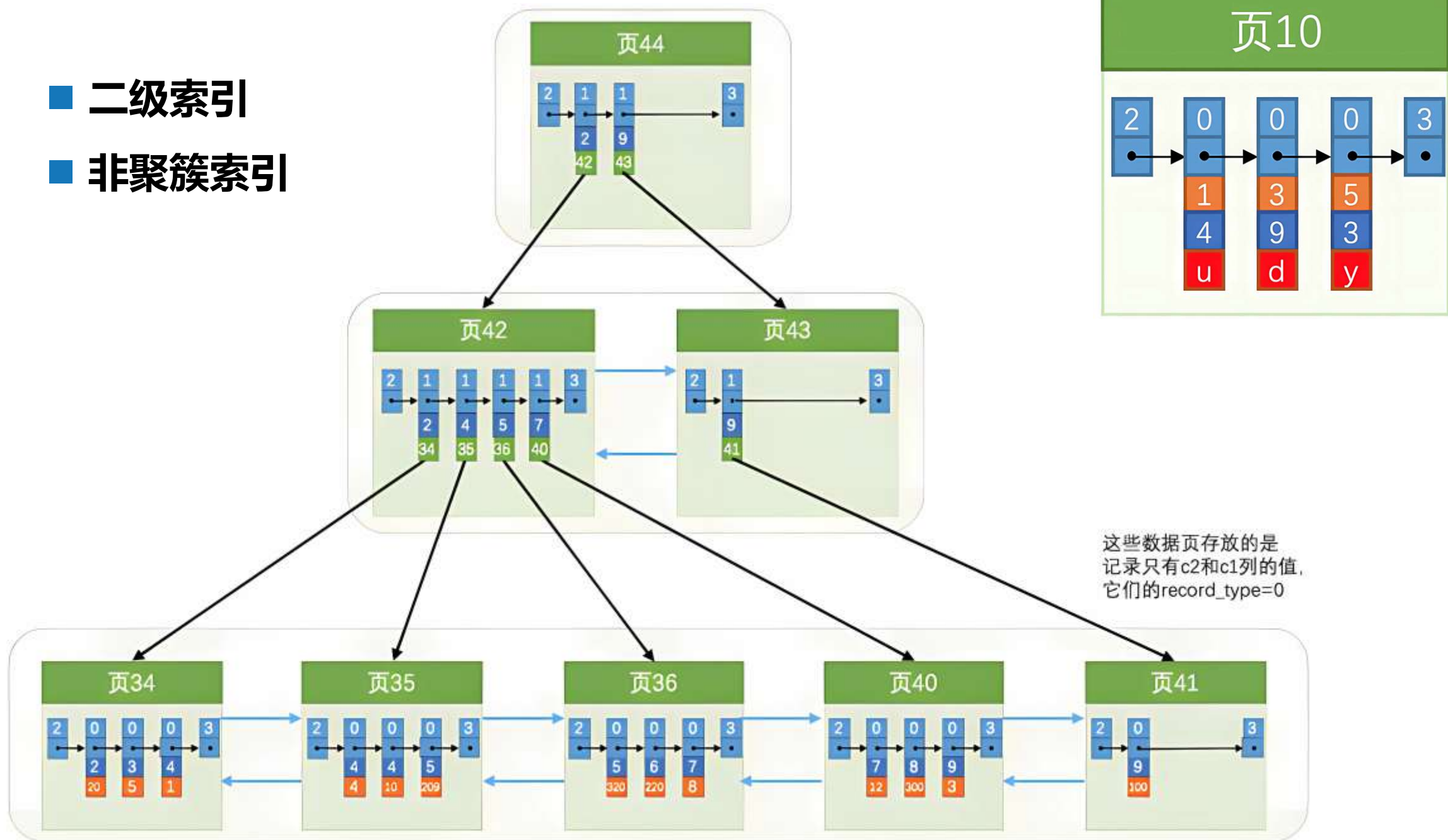
- 主索引
- B+ 树索引
- 聚簇索引



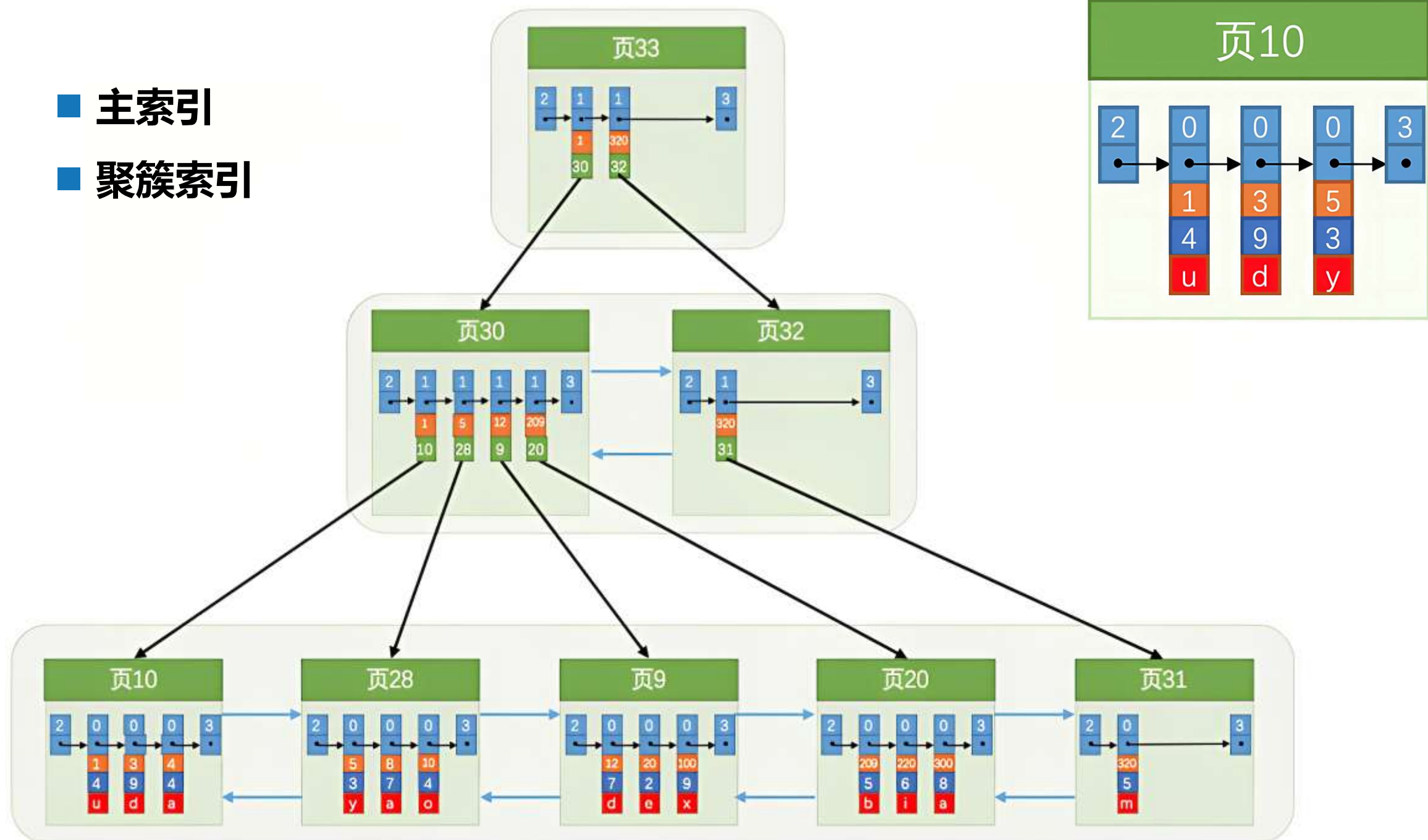
■ B+ 树的容量

- 如果 B+ 树只有 1 层，也就是只有 1 个用于存放用户记录的节点，最多能存放 100 条记录
- 如果 B+ 树有 2 层，最多能存放 $1000 \times 100 = 100000$ 条记录
- 如果 B+ 树有 3 层，最多能存放 $1000 \times 1000 \times 100 = 100000000$ 条记录
- 如果 B+ 树有 4 层，最多能存放 $1000 \times 1000 \times 1000 \times 100 = 100000000000$ 条记录
- 你的表里能存放**一千亿**条记录么？所以一般情况下，B+ 树都不会超过4层，意味着找到一条记录不会超多 4 次 I/O

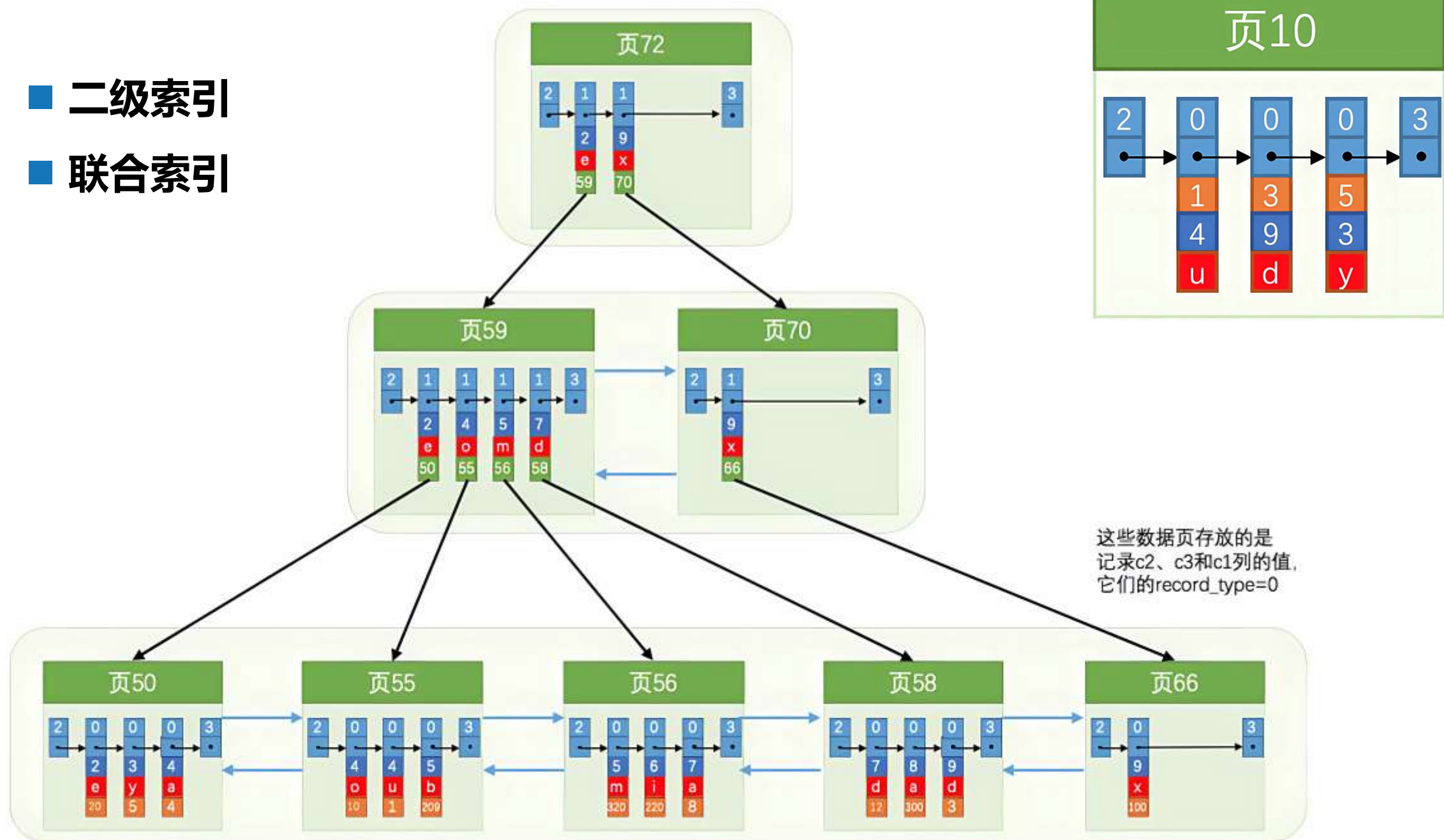
- 二级索引
- 非聚簇索引



- 主索引
- 聚簇索引



- 二级索引
- 联合索引





存储介质



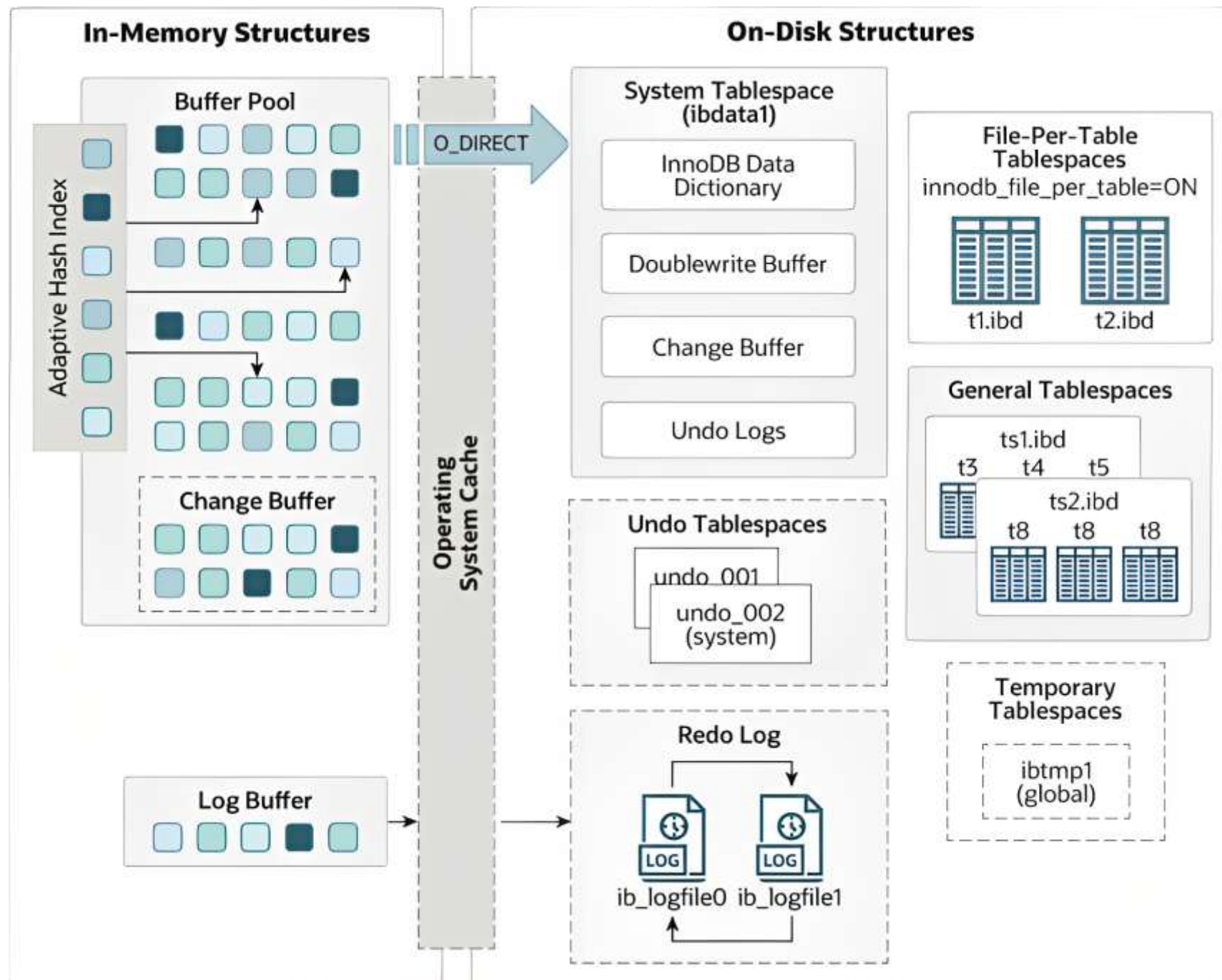
存储引擎

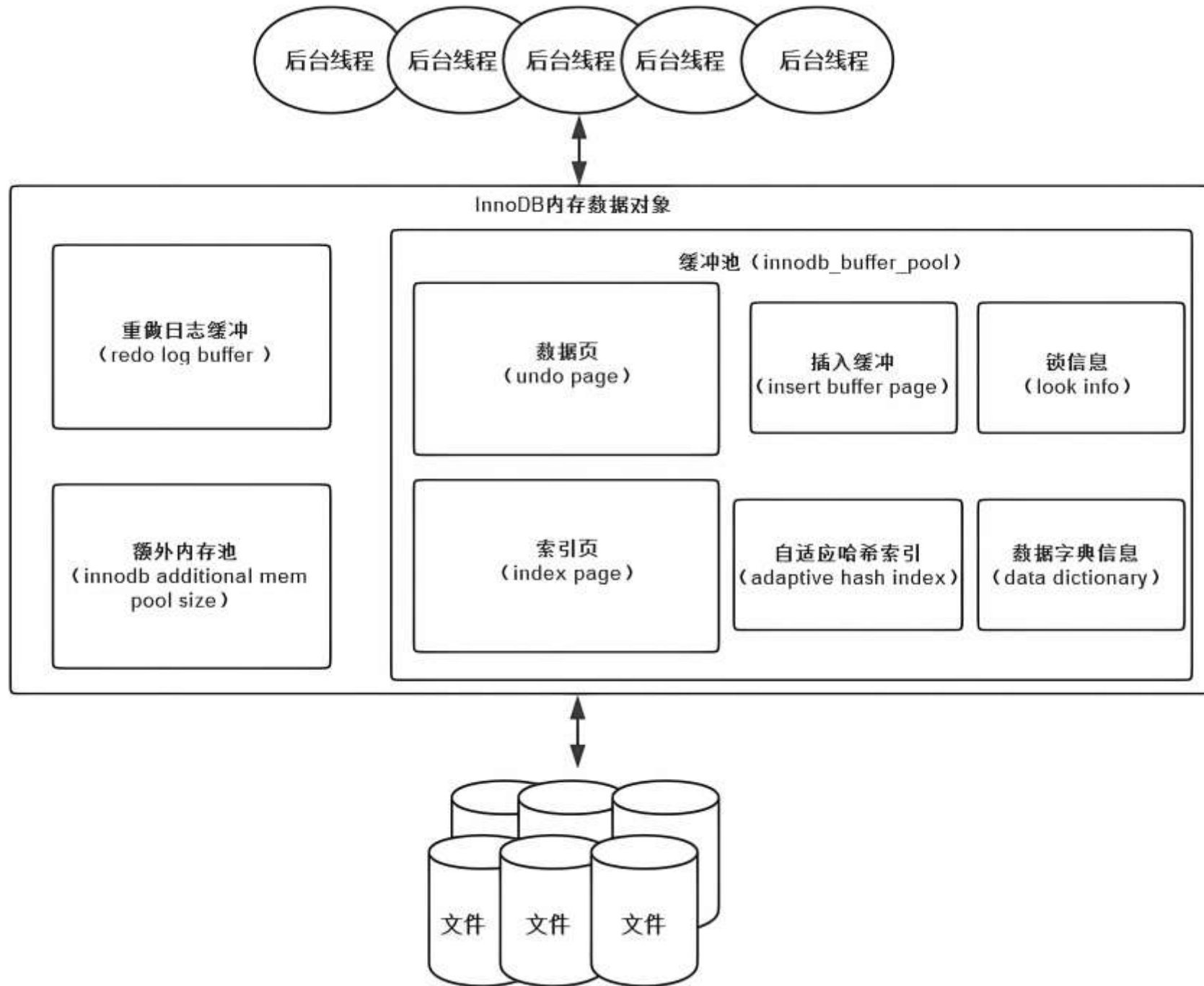


存储结构



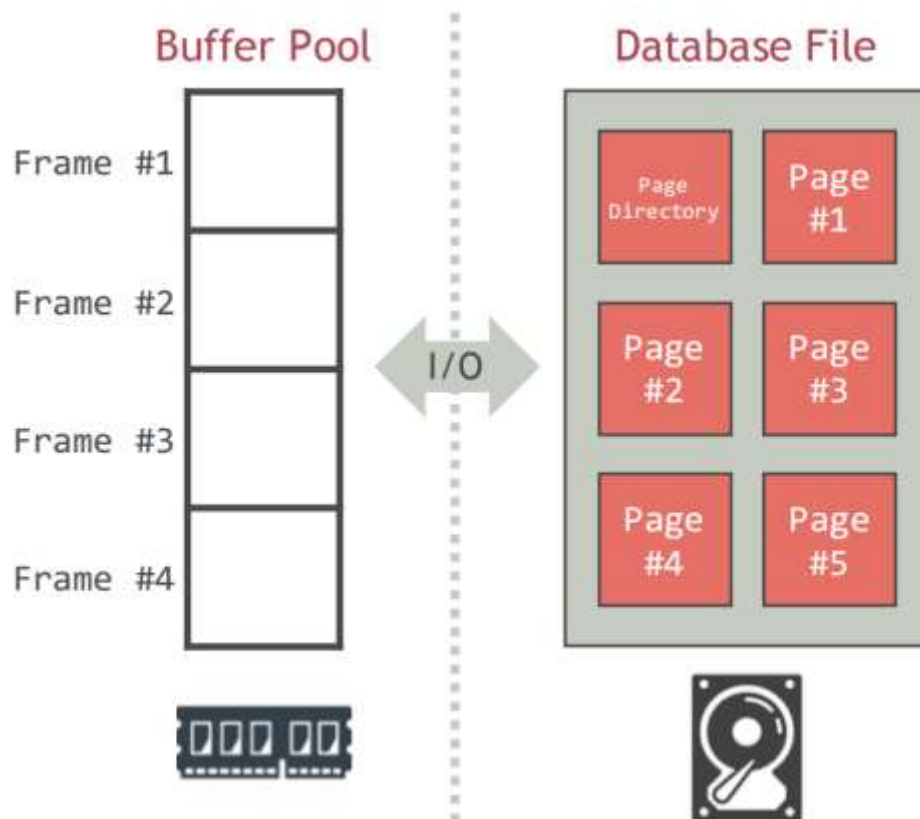
缓冲区管理





■ 缓冲池

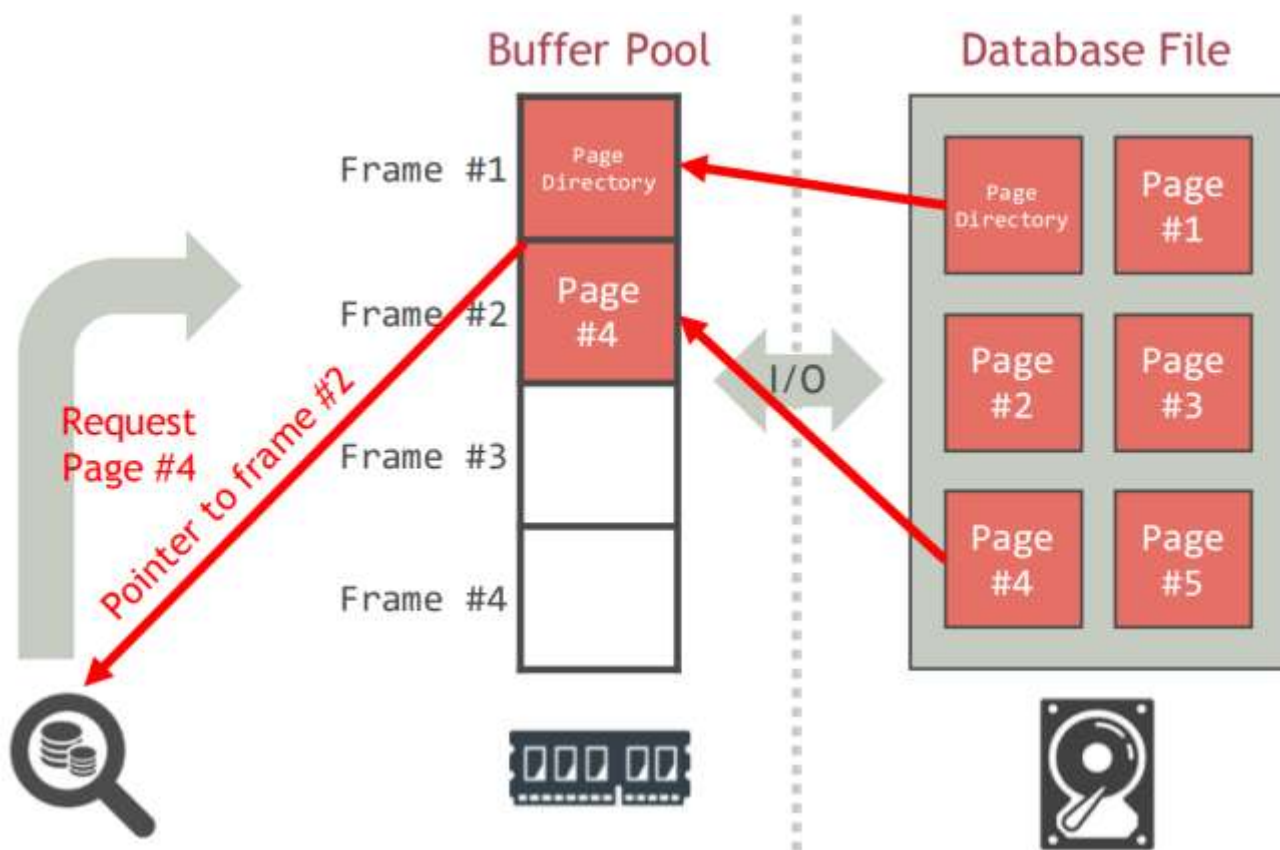
- 可用的内存区域被划分为一个固定大小页面的数组，这些页面被集体称为缓冲池
- 缓冲池中的页面称为页框



■ 缓冲区管理器

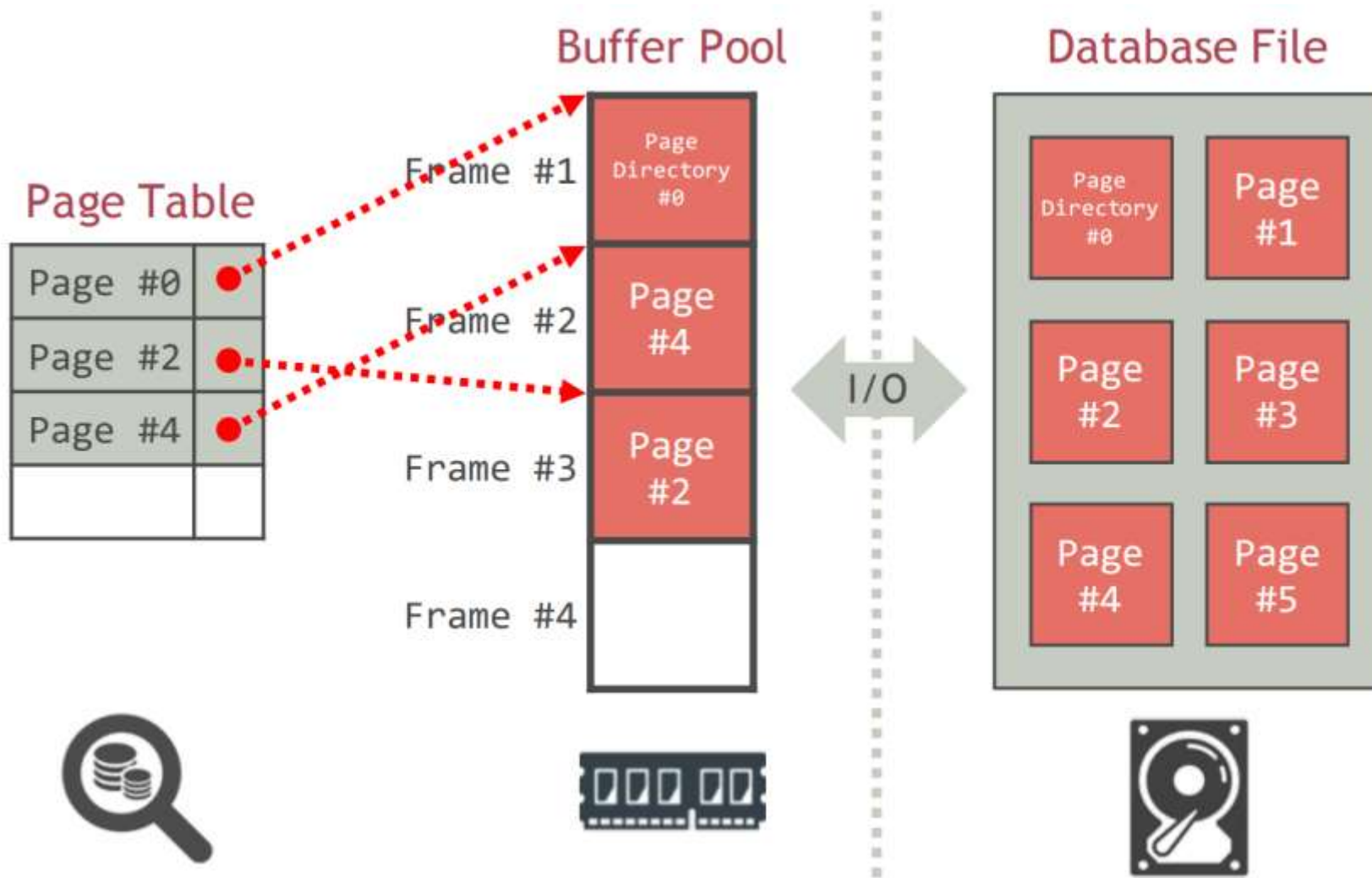
缓冲区管理器负责在需要将页带入缓冲池

- 缓冲区管理器决定要替换缓冲池中的哪个现有页面以腾出空间，以便将新页面带入（如果缓冲池已满）



■ 缓冲区管理器内部：页表

页表跟踪当前在缓冲池中的页面



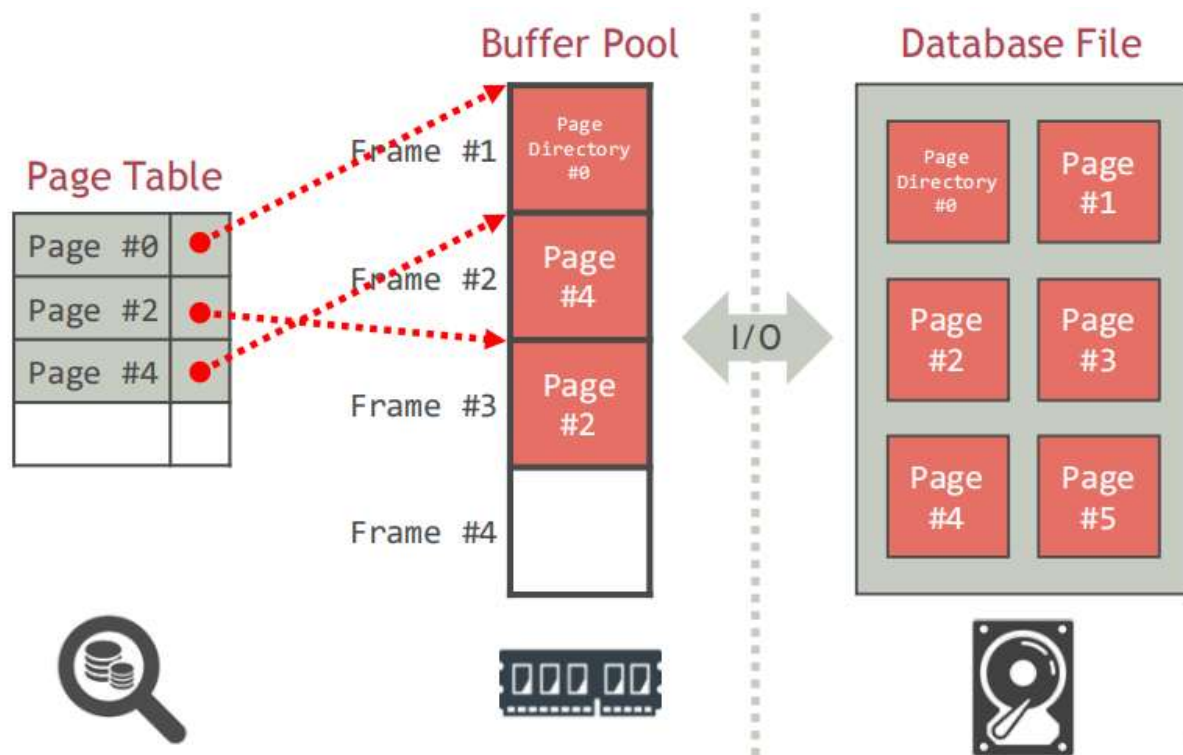
■ 页表与页目录

页目录是页面 ID 到数据库文件中的页面位置的映射

➤ 所有更改都必须记录在磁盘上，以便允许数据库管理系统在重新启动时查找

页表是页面 ID 到缓冲池帧中页面的拷贝之间的映射

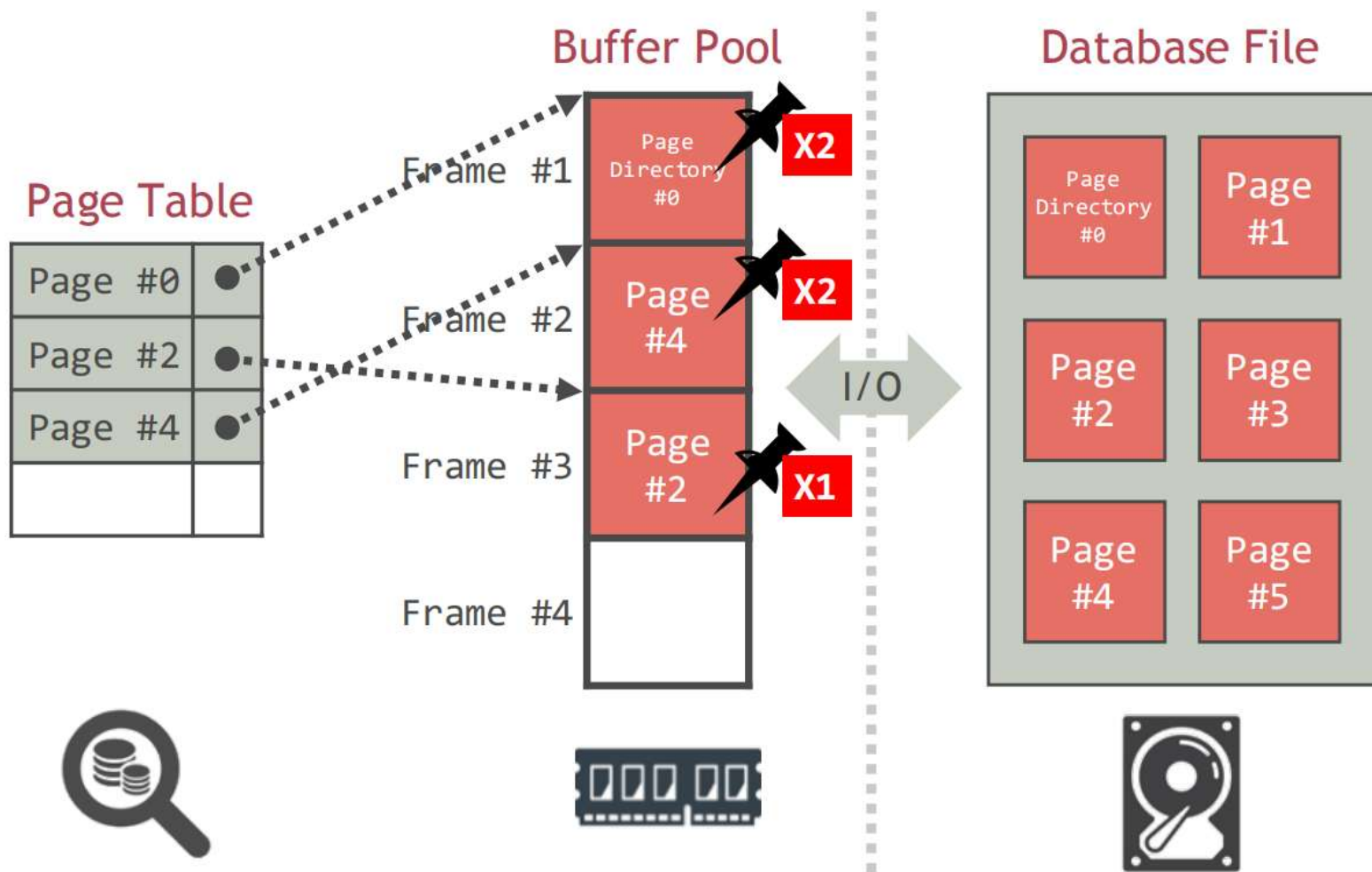
➤ 这是一个不需要存储在磁盘上的内存数据结构



■ 缓冲区管理器内部：帧的元数据

缓冲区管理器为每个帧维护两个变量

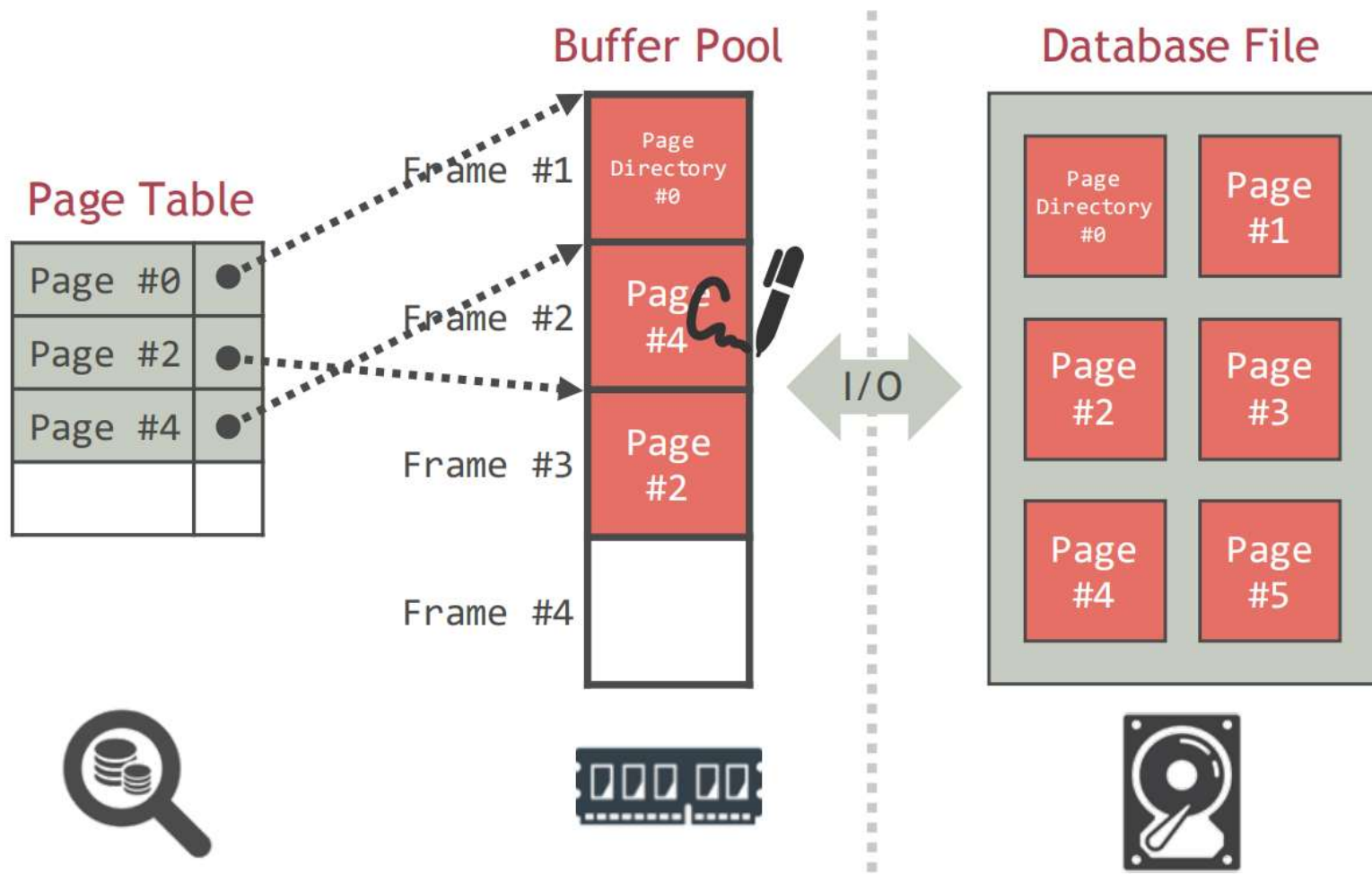
- pin_count: 当前页面在帧中已被请求但尚未释放的次数，即页面的当前用户数



■ 缓冲区管理器内部：帧的元数据

缓冲区管理器为每个帧维护两个变量

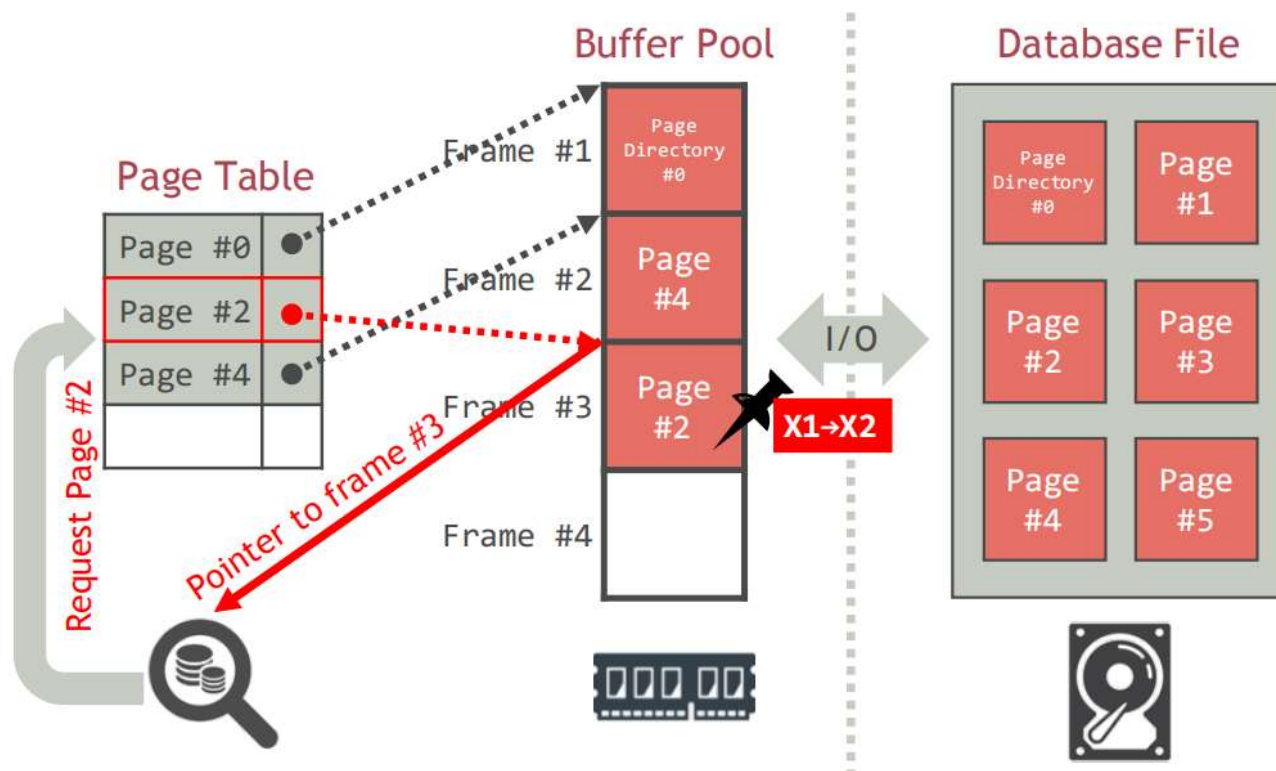
- dirty: 页面在帧中的状态，是否自从带入缓冲池以来被修改过



■ 页请求

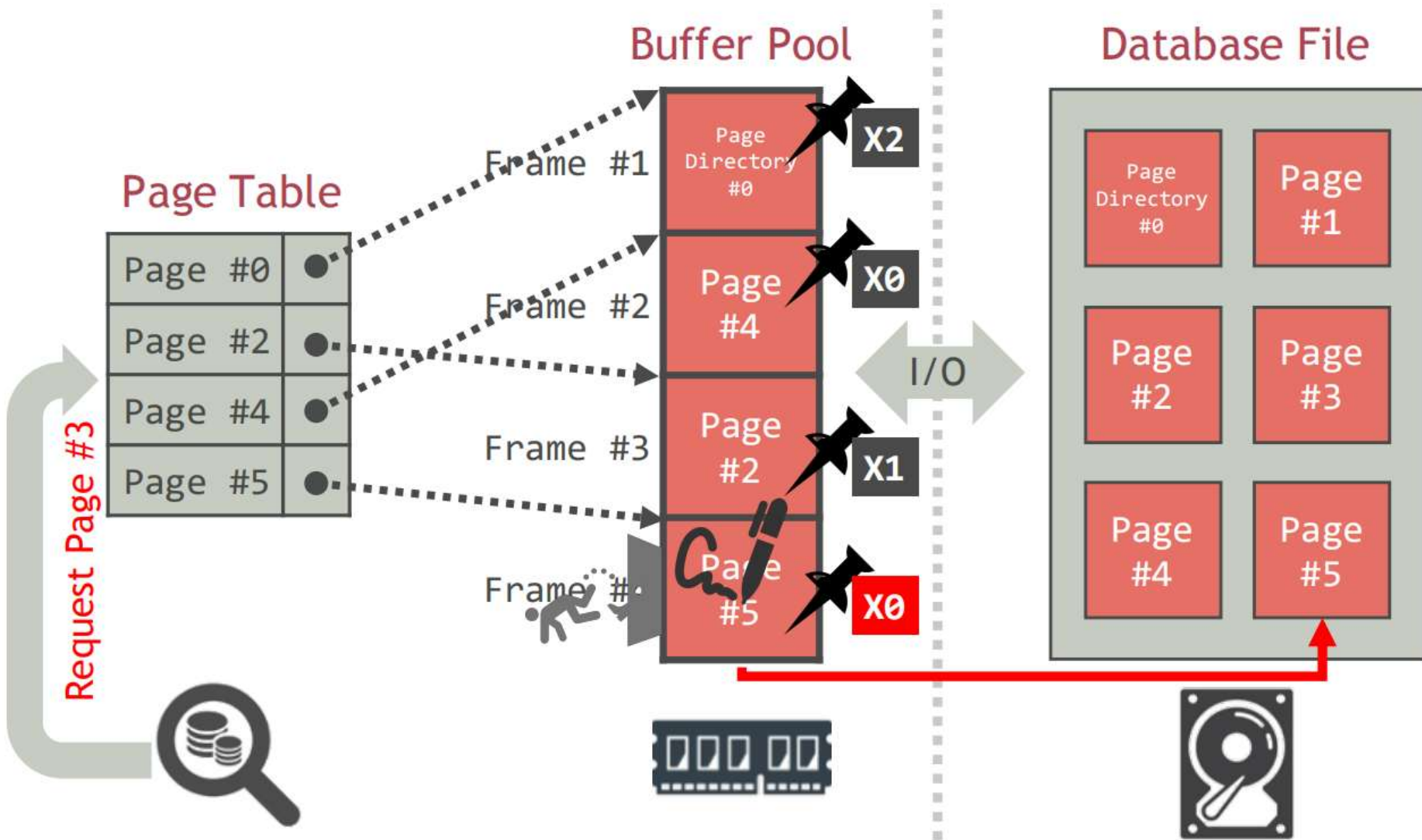
- 检查页表，找到某个帧中是否包含被请求的页 P
- 如果 P 在缓冲池中，将页 P 置为固定状态，即增加包含 P 的帧的 pin_count
- 返回包含 P 的帧的指针

Example: Request page #2



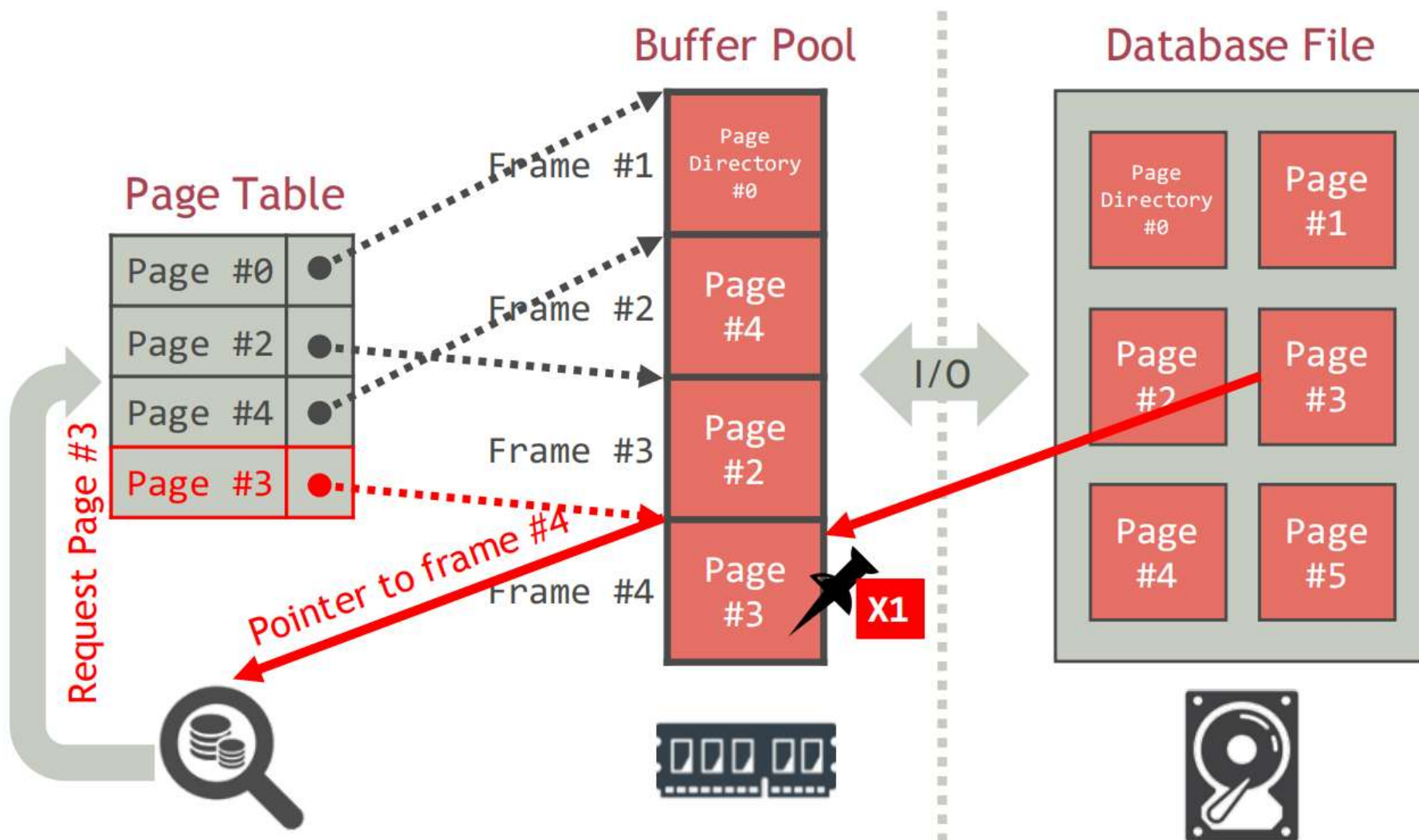
■ 页请求 (续)

Example: Request page #3



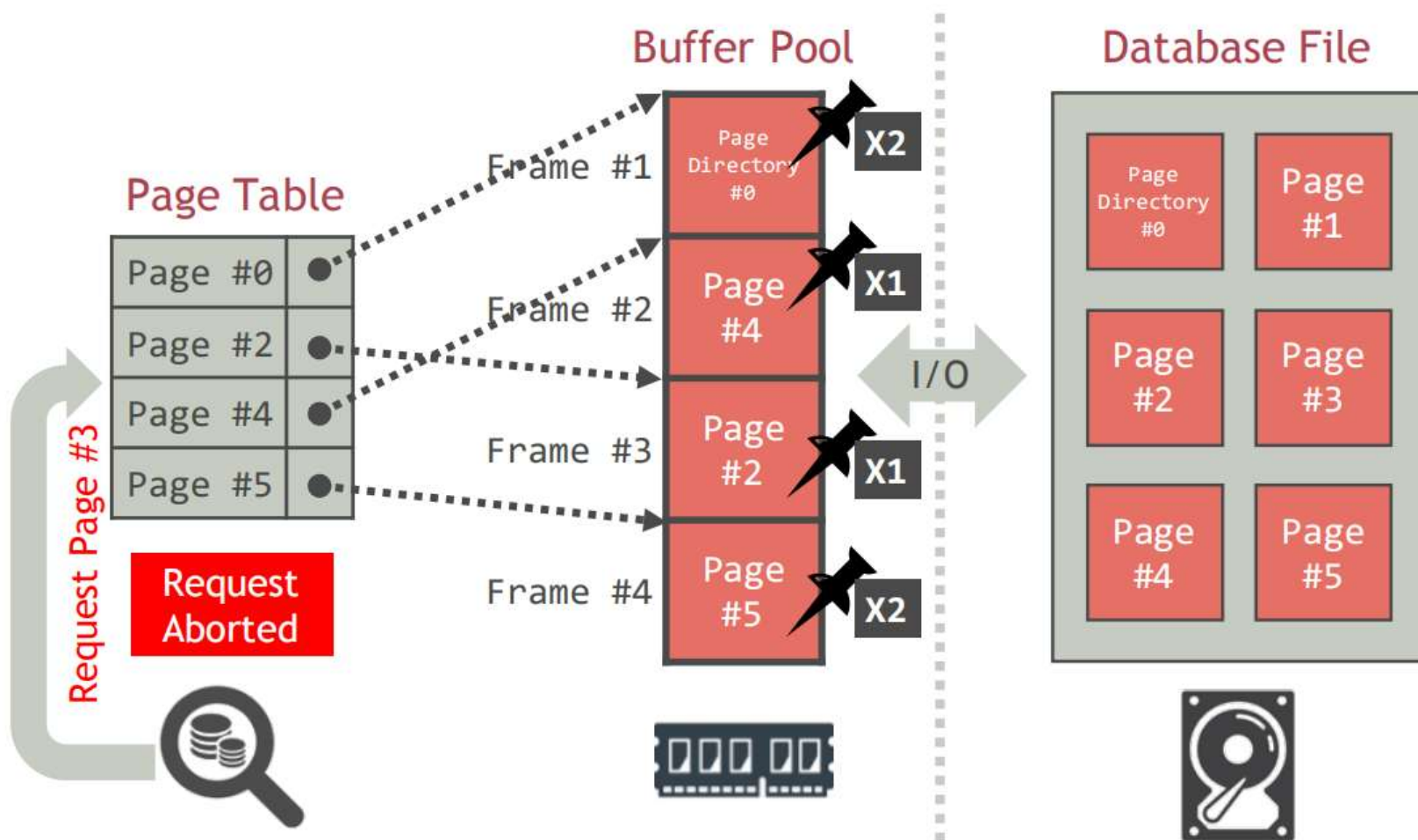
■ 页请求 (续)

Example: Request page #3



■ 页请求 (续)

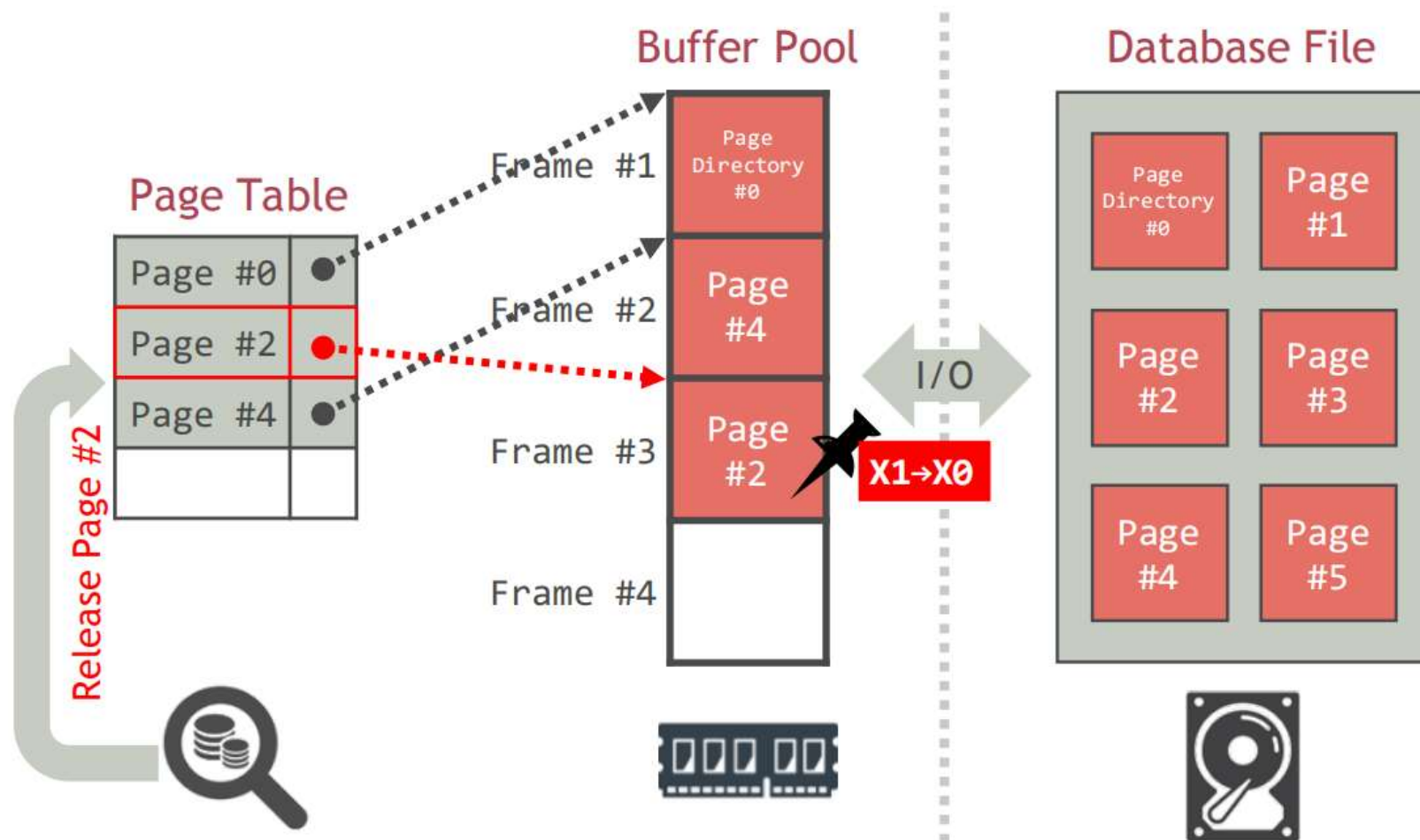
如果缓冲池中没有任何帧的 $\text{pin_count} = 0$ ，则缓冲管理器必须等待某些页被释放才能响应页请求。在实践中，可能会简单地终止请求页的事务执行



■ 页请求 (续)

解除已释放的页的固定状态，即减少包含该页的帧的 pin_count

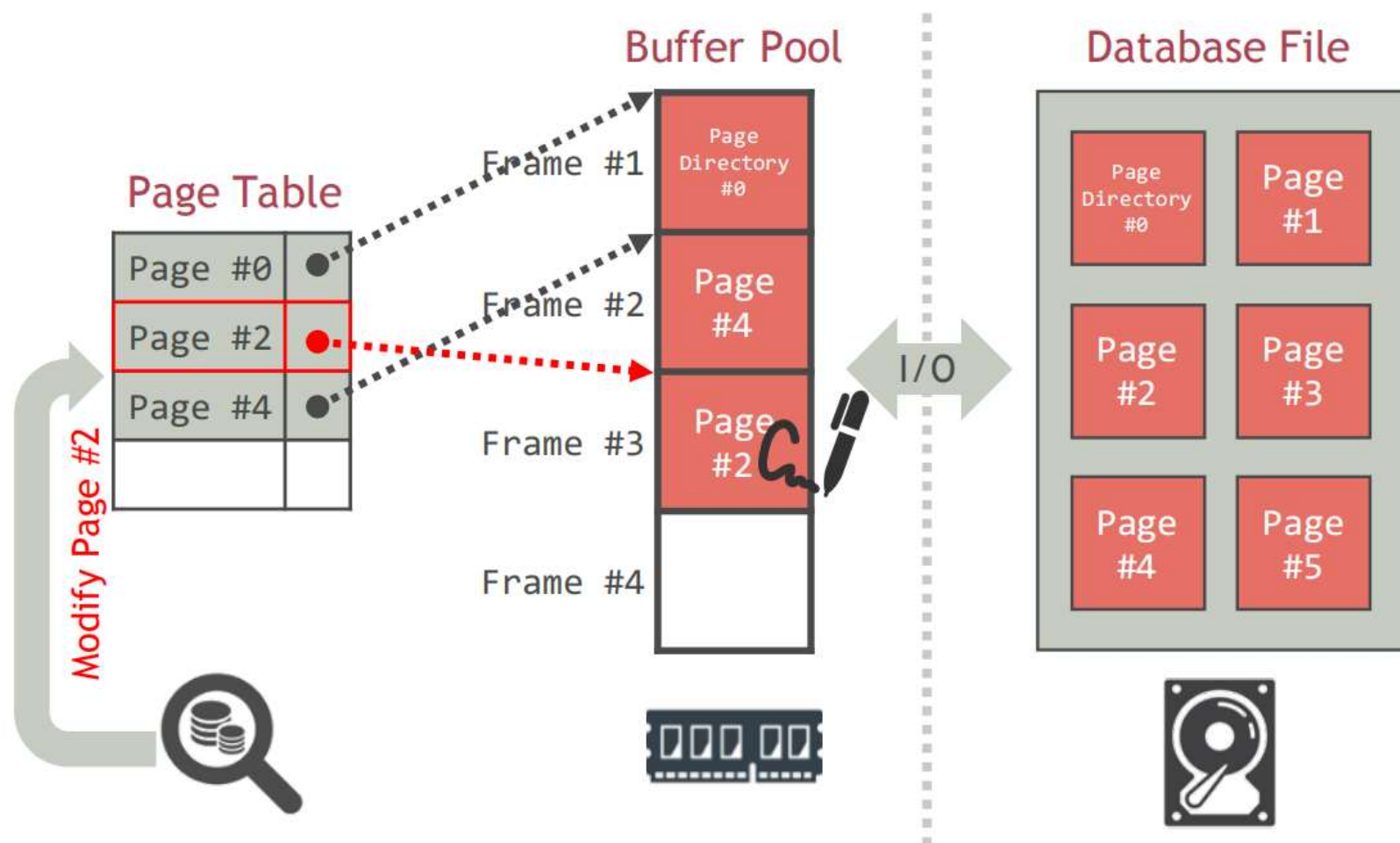
Example: Release page #2



■ 页修改

设置包含已修改页的帧的脏位

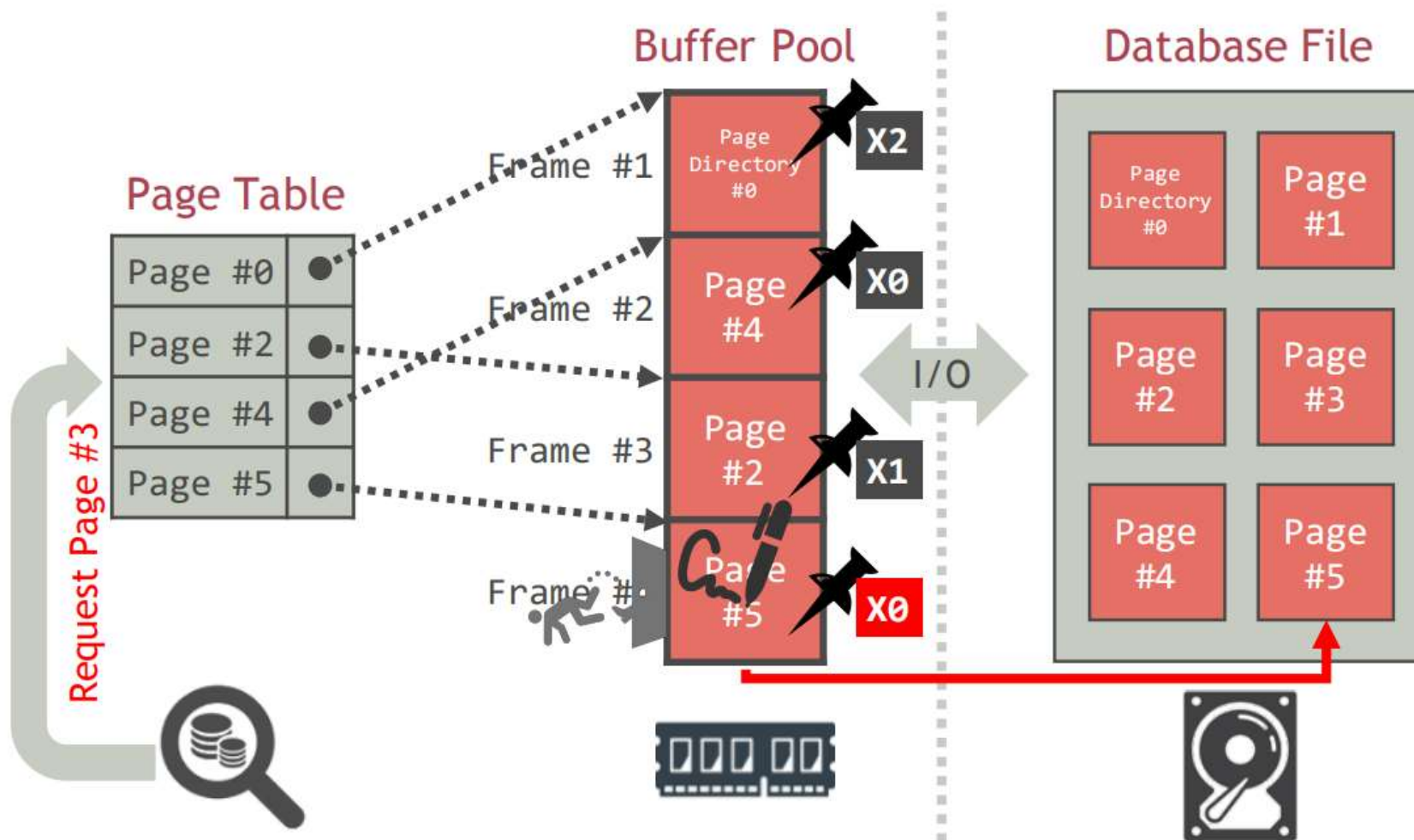
Example: Modify page #2



■ 页替换策略

当缓冲管理器需要释放帧以为新页腾出空间时，必须决定从缓冲池中驱逐哪个页

- 页替换策略可能会显著地影响数据库操作所需的时间



■ 最近最少使用（LRU）置换策略

维护每个页上次访问的时间戳

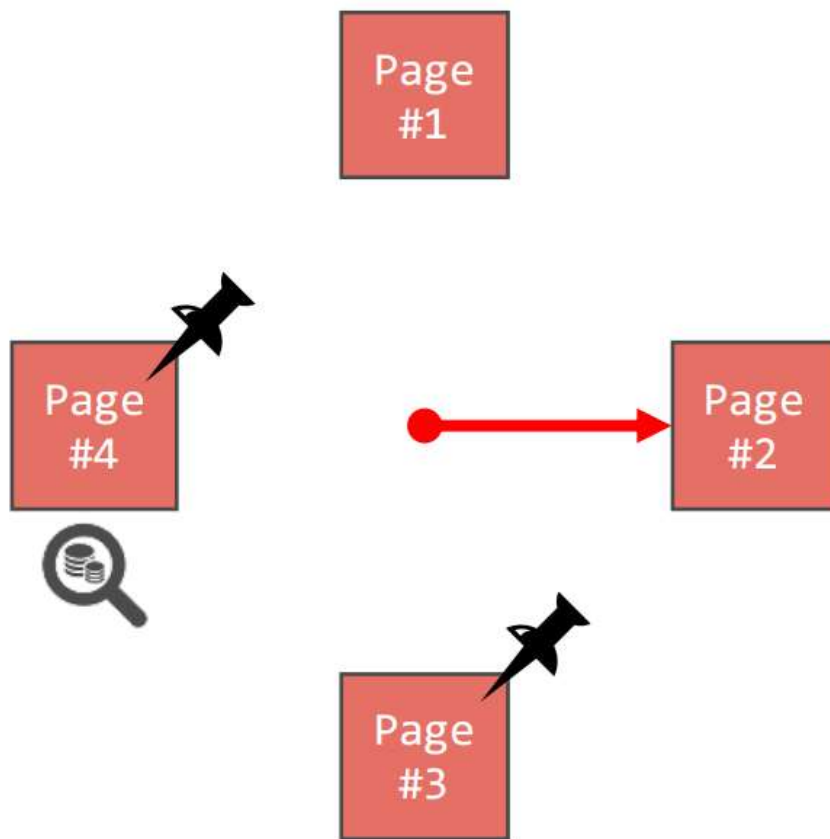
当数据库管理系统需要驱逐页时，选择时间戳最老的页

➤ 保持页面排序，以减少驱逐时的搜索时间

■ 时钟替换策略

时钟替换策略是 LRU 的一个近似方法，无需每个页面单独的时间戳

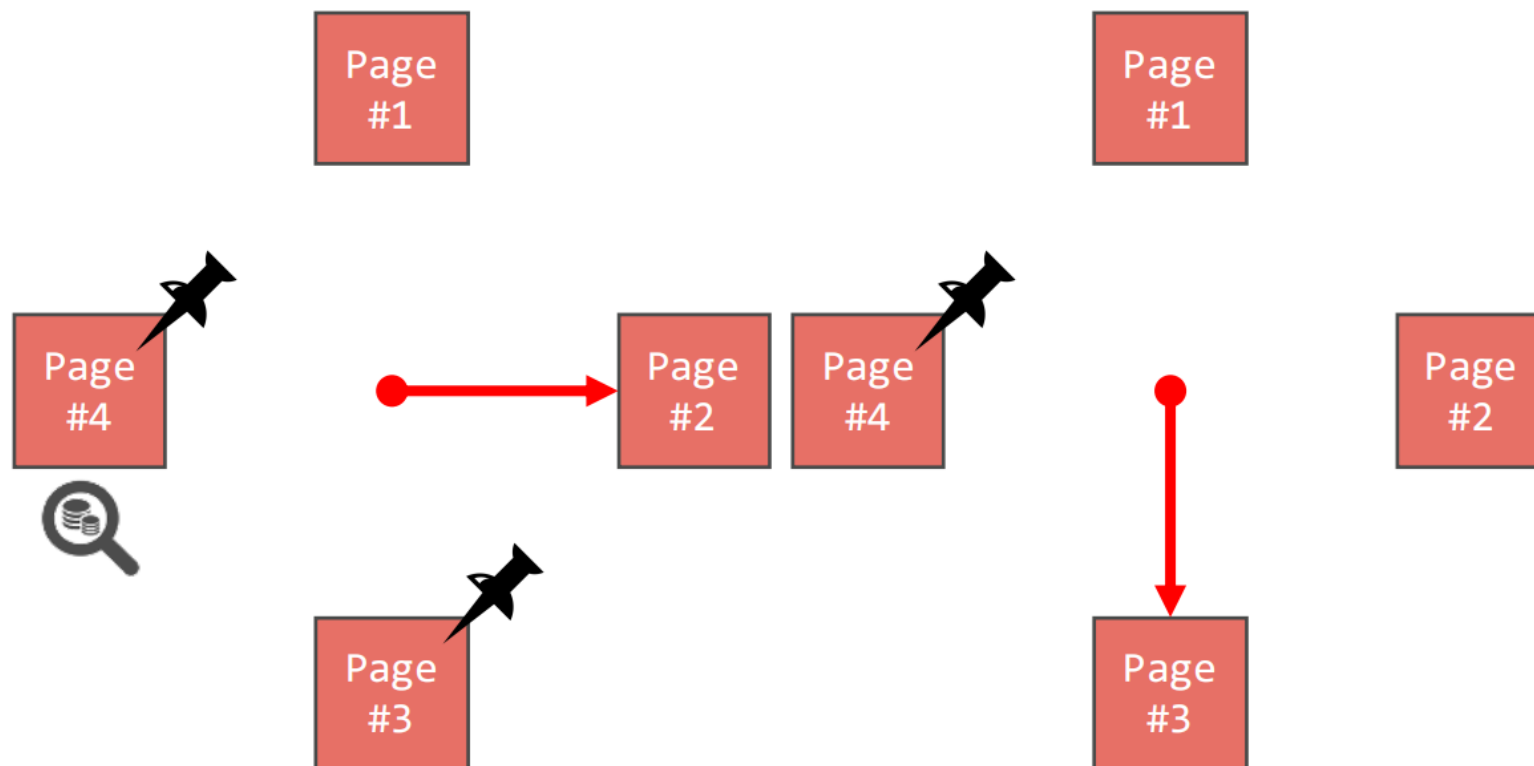
- 每个页面框架有一个引用位，它是 0 或 1
- 当一个页面被读入一个页面框架，或在页面框架中的页面被访问时，它的引用位被设置为 1



■ 时钟替换策略（续）

页面框架以循环缓冲区的形式组织，有一个“时钟手”

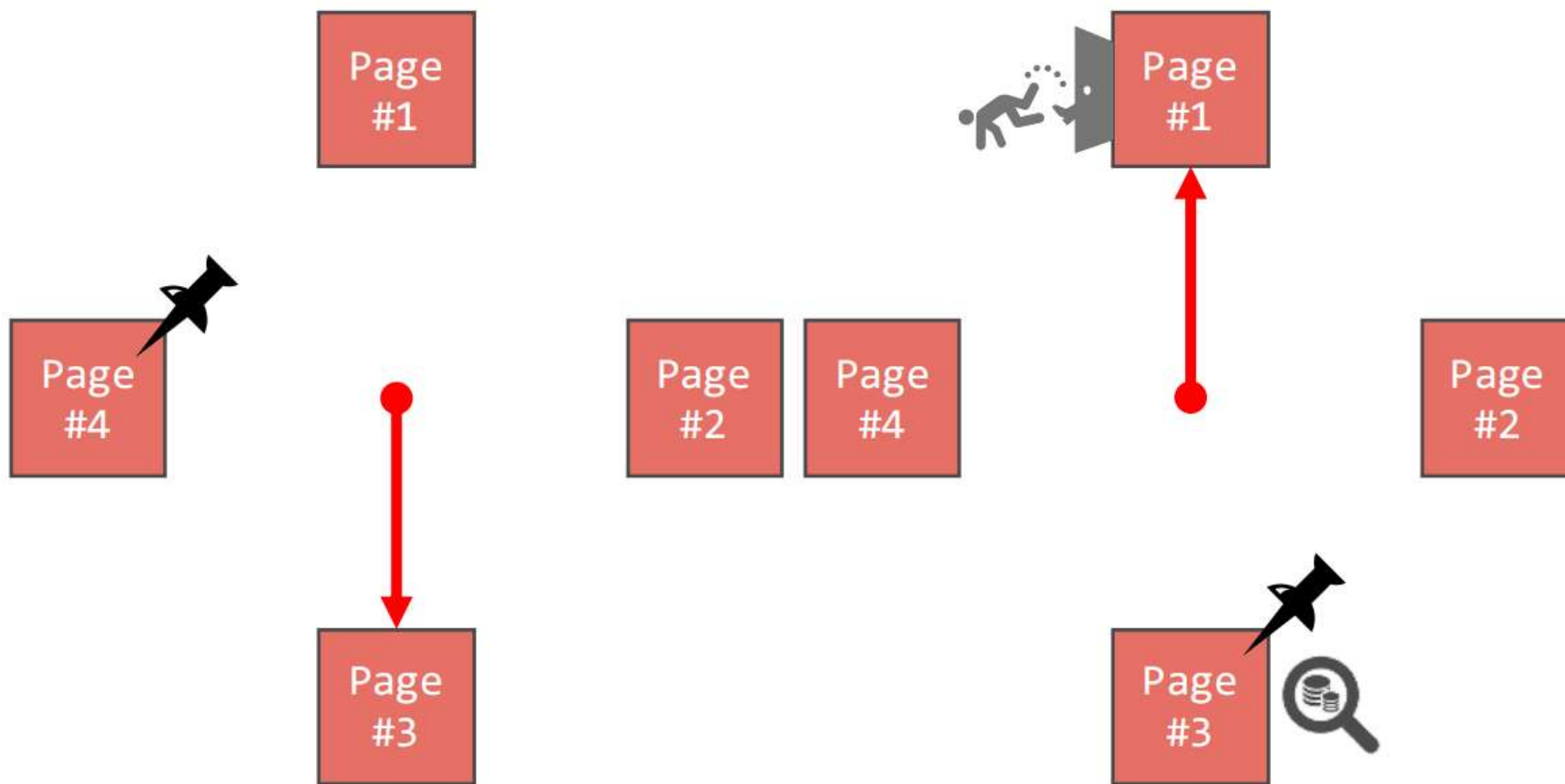
- 时钟手向顺时针方向旋转
- 扫描时，检查页面的引用位是否设置为 1
- 如果是，将引用位设置为 0



■ 时钟替换策略（续）

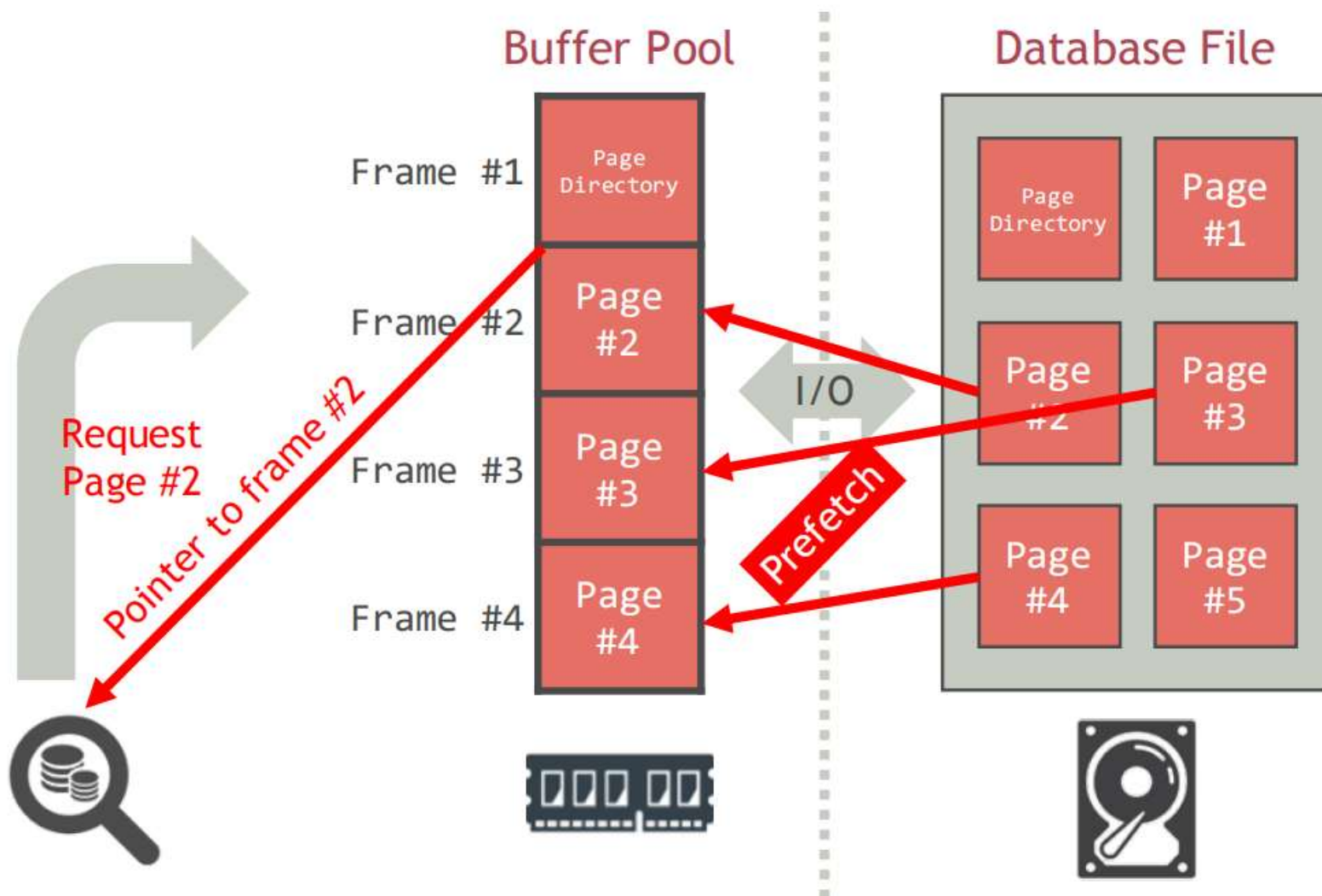
当缓冲管理器需要为新页面取一个框架时，

- 查找第一个引用位为 0 的框架
- 驱逐框架中的页面



■ 预取

为了更高效地处理页面请求，缓冲管理器经常预取即将在不久的将来请求的页面。

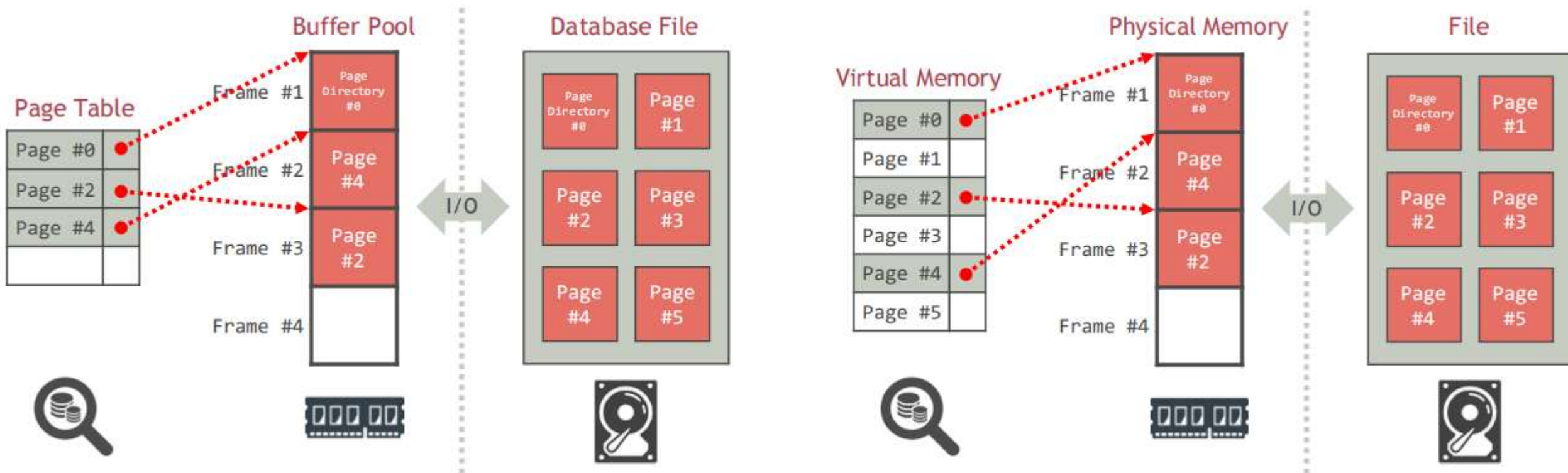


■ 缓冲池与虚拟内存

相似之处

- 目标相似：都提供对不能放入主内存的更多数据的访问
- 操作类似：都将页面从磁盘带入主内存，替换不再需要的页面

为什么不能使用操作系统的虚拟内存能力来构建 DBMS?



■ 缓冲池与虚拟内存

为什么不能使用操作系统的虚拟内存能力来构建 DBMS?

原因1 (页面引用模式预测)

DBMS 通常能够比典型的操作系统环境更准确地预测页面将被访问的顺序或页面引用模式

结果

- 页面替换：能够预测页面引用模式使得选择要替换的页面更加优化
- 页面预取：能够预测页面引用模式，使得缓冲管理器能够预测下几个页面的请求，并在请求页面之前将相应的页面提前从磁盘读入内存

■ 缓冲池与虚拟内存（续）

为什么不能使用操作系统的虚拟内存能力来构建 DBMS？

原因2（强制页面写入）

DBMS 需要能够明确地将页面强制写入磁盘，即确保磁盘上的页面副本与内存中的副本已经更新

➤ 写页面到磁盘的操作系统命令可能通过记录写请求并延迟实际修改磁盘副本来实现。如果系统在此期间崩溃，则对于DBMS 来说会产生灾难性影响。

结果

DBMS 必须能够确保在实施写前日志记录（WAL）协议用于崩溃恢复时，某些页面在缓冲池中被写入磁盘，而其他页面则在它们之前被写入。



存储介质



存储引擎



存储结构



缓冲区管理